

FinWallet Teknik ve Finansal Sözlük

Amaç

Bu belge FinWallet projesinin mimari tasarımında, teknik tasarımında, kaynak kodunda ve 00-25 numaralı proje dokümanlarında kullanılan ana teknik ve finansal terimlerin kapsamlı sözlüğüdür.

Her terim dört soruya cevap verir:

- Tanım: Terim genel olarak nedir?
- FinWallet'ta kullanım: Projede nerede ve nasıl kullanıldı?
- Neden: Hangi problemi çözmek için seçildi?
- Not / trade-off: Yanlış anlaşılabilirlik nokta, maliyet veya alternatif nedir?

Bu sözlük 314 terim içerir. "Kullanılmayan" bölümündeki terimler de dokümanlarda mimari karar gerekçesi olarak geçtiği için açıklanmıştır.

Hızlı Mimari ve Teknik Karar Haritası

Seçim / Teknik	FinWallet'ta ne için kullanıldı?	Neden seçildi?
Modular Monolith	Finansal çekirdeği tek deploy/DB transaction sınırında tutmak	Wallet/Ledger/Transaction atomikliği; distributed transaction maliyetinden kaçınma
Layered + pragmatic Clean Architecture	HTTP, use-case, domain ve infrastructure sorumluluklarını ayırmak	Kodun okunabilirliği, test edilebilirlik, teknoloji bağımsız domain
DDD-lite	Finansal dili doğrudan modele taşımak	Money, Wallet, Ledger, FraudDecision gibi iş kavramlarını doğru ifade etmek
Ports + Adapters / ACL	Fake/gerçek provider detaylarını Application/Domain'den izole etmek	Provider değişse bile core iş kurallarını korumak
YARP API Gateway	Tek public giriş, JWT, routing, rate limit, load balancing	Backend topology'yi gizlemek ve edge policy'leri merkezileştirmek
Defense in Depth	Gateway + API + internal service credential kontrollerini katmanlamak	Gateway bypass ve tek kontrol failure riskini azaltmak
MSSQL	Finansal durable source of truth olmak	ACID transaction, locking, constraints, audit/history
Redis	Sadece transient/TTL/counter state tutmak	OTP/fraud velocity gibi düşük-latency state'i hızlandırmak
Double-Entry Ledger	Her para hareketini debit/credit olarak audit etmek	Paranın kaynağını/hedefini açıklamak ve Debit=Credit invariant'ı
Wallet Current Balance	Hızlı current projection sağlamak	Her request'te tüm ledger'ı sum etmemek
Durable Idempotency	Retry/timeout'ta duplicate financial effect'i engellemek	Aynı request'in ikinci kez para hareketi üretmemesi
Outbox	DB commit ile notification/message kaydını atomik tutmak	Crash sonrası mesaj kaybını engellemek
Inbox	Provider callback duplicate'larını dedupe etmek	Aynı callback'in ikinci kez para hareketi üretmemesi
Internal + External Fraud	Server-side risk + provider risk kararını birleştirmek	Transfer/Purchase öncesi risk kontrolü
Manual Fraud Review	Şüpheli işlemi para hareketi olmadan bekletmek	Human-in-the-loop Allow/Deny
Cutoff / Business Calendar	Bank işlem tarihini iş gününe göre planlamak	After-hours/holiday bank withdrawal'larını doğru schedule etmek

Seçim / Teknik	FinWallet'ta ne için kullanıldı?	Neden seçildi?
Blocked Balance	Pending wallet->bank hareketinde parayı rezerve etmek	Aynı paranın iki kez harcanmasını engellemek
Compensation / Reversal / Refund	Committed hareketi silmeden karşıt işlem yapmak	Immutable audit trail ile düzeltme
Reconciliation	Local wallet/ledger/bank/provider kayıtlarını karşılaştırmak	Silent drift ve missing callback/mismatch tespiti
Keyset Pagination	Transaction history'yi büyük OFFSET olmadan okumak	Büyük tabloda daha stabil ve hızlı pagination
HttpClientFactory + connection pooling	Provider bağlantılarını sağlıklı reuse etmek	Socket exhaustion ve latency azaltmak
Structured Logging + Correlation ID	Dağıtık flow'u loglarda izlemek	Incident/troubleshooting ve latency analizi
Docker Compose	Tüm stack'i reproducible local/integration environment'ta çalıştırmak	Gateway + APIs + MSSQL + Redis + fake providers tek komutla
Named Volumes	MSSQL/Redis state'ini container recreate'den bağımsız tutmak	Local persistence ve backup kolaylığı
GitHub Actions CI	Build/test/Docker/PDF doğrulamasını otomatik yapmak	Regression'ı merge öncesi yakalamak
xUnit + Moq	Application davranışını hızlı unit test etmek	Business branch/failure interaction doğrulaması
Real Docker Smoke/Integration Tests	DI, config, SQL schema ve network'ü gerçek stack'te doğrulamak	'Derleniyor ama çalışmıyor' problemini yakalamak

Temel Finansal Kavramlar - Kısa Örneklerle

Wallet ile BankAccount neden ayrı?

Başlangıç:

Müşterinin FakeBank hesabı : 5.000 TRY
FinWallet Wallet : 0 TRY

Bankadan Wallet'a 1.000 TRY funding sonrası:

Müşterinin FakeBank hesabı : 4.000 TRY
FinWallet Wallet : 1.000 TRY

Wallet, bankadaki aynı satırın kopyası değildir. BankAccount dış bankadaki hesabı; Wallet ise FinWallet içindeki müşteriye ait kullanılabilir bakiyeyi ve FinWallet'ın müşteriye karşı yükümlülüğünü temsil eder.

Ledger nedir?

Ledger, para hareketlerinin "neden ve hangi hesaplar arasında" oluştuğunu debit/credit kayıtlarıyla açıklayan immutable muhasebe geçmişidir. Wallet tablosu bugün kullanılabilir bakiye kaç sorusuna hızlı cevap verir; Ledger bu bakiyenin nasıl oluştuğunu açıklar.

Bankadan Wallet'a 1.000 TRY örneği:

Debit	BANK-SETTLEMENT:TRY	1.000
Credit	WALLET-LIABILITY:<walletId>	1.000

Toplam Debit		1.000
Toplam Credit		1.000

Ekonomik anlam:

- Settlement Asset +1.000: FinWallet'ın bankacılık tarafındaki karşılık varlığı arttı.
- Customer Wallet Liability +1.000: FinWallet artık müşteriye 1.000 TRY wallet bakiyesi borçlu.

Wallet-to-Wallet Transfer örneği

Ali'den Ayşe'ye 300 TRY:

Debit	WALLET-LIABILITY:ALI	300
Credit	WALLET-LIABILITY:AYSE	300

Debit = Credit		

Source wallet liability debit edilerek azalır; destination liability credit edilerek artar. Banka çağrısı gerekmez, çünkü bu FinWallet içi book transfer'dır.

Purchase + Platform Kampanyası örneği

Ürün 200 TRY, platform 20 TRY indirim sponsor ediyor, müşteri 180 TRY ödüyor:

Debit	WALLET-LIABILITY:CUSTOMER	180
Debit	CAMPAIGN-EXPENSE	20
Credit	MERCHANT-PAYABLE	200

Toplam Debit		200
Toplam Credit		200

Bu kayıt "20 TRY nereden geldi?" sorusunu açıkça cevaplar: platform expense olarak üstlendi.

Refund / Reversal neden UPDATE değildir?

Original transaction veya journal silinmez/değiştirilmez. Yeni bir karşıt FinancialTransaction ve yeni bir journal oluşturulur.

```
Original Purchase
|
+--> immutable history

Refund
|
+--> yeni transaction
+--> original etkisini tersleyen yeni journal
```

Böylece audit trail korunur.

Idempotency neden finansal sistemde kritiktir?

Client transfer request'ini gönderdi, server parayı hareket ettirdi fakat response network'te kayboldu. Client tekrar denerse aynı para ikinci kez hareket etmemelidir.

```
Idempotency-Key: transfer-abc-001

1. request -> işlem Completed
2. aynı key + aynı payload -> eski sonuç replay
3. aynı key + farklı payload -> Conflict
```

Outbox / Inbox neden var?

```
Outbox:
Financial DB COMMIT
|
+--> Outbox message aynı transaction'da kaydedilir
|
+--> Worker daha sonra SMS/notification gönderir

Inbox:
Provider callback
|
+--> Source + MessageId ile dedupe
|
+--> aynı callback 100 kez gelse de finansal effect tekrar uygulanmaz
```

Reconciliation neden otomatik düzeltme yapmıyor?

Reconciliation bir "fark bulma" mekanizmasıdır, sessiz bir balance overwrite mekanizması değildir.

```
Wallet current balance
vs
Ledger-derived balance
```

Fark varsa ReconciliationIssue oluşturulur. Para geçmişini otomatik değiştirilmez.

Sözlük

1. Mimari ve Tasarım (32 terim)

Adapter

- Tanım: Bir port'u belirli teknoloji/protokole bağlayan concrete implementasyondur.
- FinWallet'ta kullanım: FakeBank HTTP client adapter'ı IBankProvider'ı REST çağrılarını map eder.
- Neden / çözdüğü problem: Provider sözleşmesi değişse bile Application'ın değişmemesini sağlar.
- Not / trade-off: Adapter içinde validation ve DTO mapping yapılmalıdır; domain'e provider tipi taşınmamalıdır.

Aggregate

- Tanım: Birlikte tutarlı kalması gereken entity/value object grubunun transaction sınırını temsil eden DDD kavramıdır.
- FinWallet'ta kullanım: FinWallet ağır aggregate framework'ü kullanmaz; ancak Wallet/Transaction/Ledger posting kuralları aynı consistency boundary içinde ele alınır.
- Neden / çözdüğü problem: Hangi state'in birlikte atomik değişmesi gerektiğini düşünmeye yardımcı olur.
- Not / trade-off: FinWallet'ta aggregate kavramı hafif uygulanır; her tablo ayrı aggregate değildir.

Aggregate Root

- Tanım: Bir aggregate'e dışarıdan erişilen ana entity'dir.
- FinWallet'ta kullanım: FinWallet'ta bu terim yoğun framework kuralı olarak uygulanmaz; örneğin Wallet davranışı kendi kimliği ve kuralları etrafında korunur.
- Neden / çözdüğü problem: Aggregate içindeki kurallara kontrollü giriş sağlar.
- Not / trade-off: Projede resmi aggregate-root hierarchy kurulmadığı için 'hafif kullanılan' bir terimdir.

Anti-Corruption Layer (ACL)

- Tanım: Dış sistemin modelini iç domain modelinden ayıran dönüştürme katmanıdır.
- FinWallet'ta kullanım: Bank, Fraud, Cutoff ve Campaign provider response'ları Infrastructure'da FinWallet tiplerine dönüştürülür.
- Neden / çözdüğü problem: Dış sistemin isimlendirme, enum ve hata davranışının domain'i kirletmesini önler.
- Not / trade-off: Ek mapping kodu üretir ama entegrasyon bağımlılığını ciddi azaltır.

Append-Only

- Tanım: Yeni history kayıtlarının eklendiği, eski kayıtların overwrite/delete edilmediği veri modeli yaklaşımıdır.
- FinWallet'ta kullanım: Ledger journal/entry ve correction history bu prensibe yakındır.
- Neden / çözdüğü problem: Audit trail ve finansal geçmişin sonradan değiştirilmemesini sağlar.

- Not / trade-off: Current-state tabloları append-only olmak zorunda değildir; history ve projection rolleri ayrıdır.

Audit Trail

- Tanım: Bir business/financial state'in hangi olaylar ve kimliklerle oluştuğunu geriye dönük izleyebilme kayıdır.
- FinWallet'ta kullanım: FinancialTransaction, Ledger, FraudEvent, reconciliation ve review metadata birlikte audit trail oluşturur.
- Neden / çözdüğü problem: Incident, dispute ve finansal investigation sırasında açıklanabilirlik sağlar.
- Not / trade-off: Audit log ile application debug log aynı şey değildir; audit kayıtları daha güçlü retention/integrity gerektirebilir.

Bounded Context

- Tanım: Aynı kelimelerin aynı anlamı taşıdığı iş alanı sınırıdır.
- FinWallet'ta kullanım: FinWallet finansal çekirdeği ile FakeBank/FakeFraud/FakeCampaign gibi dış provider modelleri ayrı anlam sınırları olarak ele alınır.
- Neden / çözdüğü problem: Provider DTO'larının doğrudan domain modeline sızmasını engeller.
- Not / trade-off: V1'de bunlar ayrı microservice bounded-context organizasyonu değil; kavramsal sınırlar ve adapter'larla korunur.

Clean Architecture

- Tanım: Bağımlılıkların dış katmanlardan içteki iş kurallarına doğru aktığı mimari prensipler bütünüdür.
- FinWallet'ta kullanım: Domain hiçbir framework/SQL/HTTP detayını bilmez; Application interface'leri tanımlar, Infrastructure bunları uygular.
- Neden / çözdüğü problem: İş kurallarını teknolojiden bağımsız tutar ve test edilebilirliği artırır.
- Not / trade-off: FinWallet tam akademik Clean Architecture değil; sadeleştirilmiş/pragmatik bir uygulamadır.

Consistency Boundary

- Tanım: Bir işlemde birlikte doğru kalması gereken verilerin sınırıdır.
- FinWallet'ta kullanım: Wallet balance, FinancialTransaction, Ledger, durable idempotency ve Outbox gerektiğinde aynı MSSQL transaction'a alınır.
- Neden / çözdüğü problem: Bir kısmı commit olup diğer kısmı kalırsa oluşacak finansal tutarsızlığı önler.
- Not / trade-off: External provider state bu sınırın dışındadır; o nedenle compensation/reconciliation gerekir.

DDD-lite

- Tanım: DDD'nin değerli modelleme fikirlerini kullanıp ağır süreç ve framework katmanlarını eklemeyen pragmatik uygulamadır.
- FinWallet'ta kullanım: Value Object, entity, invariant ve domain isimlendirmeleri kullanılır; tam Event Sourcing veya kapsamlı domain event bus kullanılmaz.
- Neden / çözdüğü problem: Finansal domain'i anlaşılır tutarken over-engineering'i sınırlar.
- Not / trade-off: Daha karmaşık domainlerde ileride daha güçlü DDD araçları gerekebilir.

Defense in Depth

- Tanım: Tek bir güvenlik kontrolüne güvenmek yerine birden fazla katmanda kontrol uygulanmasıdır.
- FinWallet'ta kullanım: Gateway JWT kontrolü yaparken FinWallet.Api de JWT/session/ownership ve service-key kontrolü yapar.
- Neden / çözdüğü problem: Bir katman bypass edilse bile sonraki katmanın isteği durdurmasını sağlar.
- Not / trade-off: Aynı kontrolü körlemesine kopyalamak değil, farklı tehdit seviyelerine uygun katmanlar kurmaktır.

Dependency Injection (DI)

- Tanım: Bir sınıfın ihtiyaç duyduğu bağımlılıkların dışarıdan verilmesidir.
- FinWallet'ta kullanım: ASP.NET Core built-in DI ile handler, store, provider, TimeProvider ve background service'ler kaydedilir.
- Neden / çözdüğü problem: Bağımlılık oluşturma işini sınıflardan çıkarır; test ve runtime wiring'i yönetilebilir yapar.
- Not / trade-off: Eksik registration compile sırasında değil startup/ilk resolve sırasında hata verebilir; bu yüzden runtime smoke test önemlidir.

Dependency Inversion Principle (DIP)

- Tanım: Üst seviye iş kurallarının alt seviye teknik detaylara değil soyutlamalara bağlı olması prensibidir.
- FinWallet'ta kullanım: Application IBankProvider, store ve fraud provider interface'lerine bağlıdır; Infrastructure concrete implementasyonları sağlar.
- Neden / çözdüğü problem: Fake provider ile gerçek provider değişimini ve unit test mock'larını kolaylaştırır.
- Not / trade-off: Her sınıf için gereksiz interface üretmek değildir; sadece gerçek boundary'lerde kullanılır.

Dependency Rule

- Tanım: İç katmanların dış katmanları bilmemesi kuralıdır.
- FinWallet'ta kullanım: Api -> Application -> Domain ve Infrastructure -> Application/Domain yönü korunur.
- Neden / çözdüğü problem: Domain'in SQL, Redis veya provider DTO'larına bağlanmasını önler.
- Not / trade-off: Kural ihlal edilirse teknoloji değişikliği domain'i kırar.

Domain-Driven Design (DDD)

- Tanım: Yazılım modelini iş alanındaki gerçek kavramlar ve kurallar etrafında kurma yaklaşımıdır.
- FinWallet'ta kullanım: Wallet, Money, Currency, FinancialTransaction, Ledger, FraudDecision gibi terimler doğrudan domain dilidir.
- Neden / çözdüğü problem: Kod ile finans/bankacılık dili arasındaki anlam farkını azaltır.
- Not / trade-off: FinWallet 'DDD-lite' kullanır; ağır aggregate/event-sourcing altyapısı kurulmamıştır.

Entity

- Tanım: Kimliği olan ve zaman içinde state değiştirebilen domain nesnesidir.
- FinWallet'ta kullanım: Wallet, BankAccount ve FinancialTransaction kimlikleriyle takip edilen entity örnekleridir.
- Neden / çözdüğü problem: Aynı kimliğin yaşam döngüsü ve kurallarını tek modelde tutar.

- Not / trade-off: Entity eşitliği sadece property değerleriyle değil kimlikle ilgilidir.

Handler

- Tanım: Bir use-case komutunu/query'sini orkestre eden Application sınıfıdır.
- FinWallet'ta kullanım: ExecuteWalletTransferHandler, ExecuteFraudProtectedPurchaseHandler gibi sınıflar bu rolü taşır.
- Neden / çözdüğü problem: Controller'ı ince tutar ve business flow'u HTTP'den ayırır.
- Not / trade-off: MediatR kullanılmadan doğrudan DI ile çağrılır.

Horizontal Scaling

- Tanım: Aynı servisin daha fazla instance/replica çalıştırılarak kapasitesinin artırılmasıdır.
- FinWallet'ta kullanım: YARP cluster'larına FinWallet.Api veya provider replica'ları eklenebilir.
- Neden / çözdüğü problem: Tek instance bottleneck'ini azaltır.
- Not / trade-off: Correctness instance sayısına değil DB constraints/idempotency/concurrency kurallarına dayanmalıdır.

Invariant

- Tanım: Sistemde hangi akış olursa olsun bozulmaması gereken iş kuralıdır.
- FinWallet'ta kullanım: Debit == Credit, negatif wallet bakiyesi yok, aynı correction iki kez uygulanamaz gibi kurallar invariant'tır.
- Neden / çözdüğü problem: Finansal doğruluğu sadece endpoint akışına değil veri/model seviyesine bağlar.
- Not / trade-off: Config ile açılıp kapanan seçenek değildir; değişmesi veri/muhasebe migrasyonu gerektirebilir.

Layered Architecture

- Tanım: Sorumlulukların katmanlara ayrıldığı mimari yaklaşımdır.
- FinWallet'ta kullanım: FinWallet.Api, Application, Domain ve Infrastructure ayrı projelerdir.
- Neden / çözdüğü problem: HTTP, use-case, domain kuralı ve teknik altyapı kodunun birbirine karışmasını engeller.
- Not / trade-off: Katmanlar gereğinden fazla soyutlanırsa over-engineering oluşabilir; projede sade tutulmuştur.

Modular Monolith

- Tanım: Tek deploy edilebilen uygulama içinde domain alanlarının modüllerle ayrıldığı mimari stildir.
- FinWallet'ta kullanım: FinWallet'ın finansal çekirdeği Wallet, BankAccount, Transaction, Ledger, Fraud, Reconciliation gibi modülleri aynı uygulama ve MSSQL transaction sınırı içinde tutar.
- Neden / çözdüğü problem: Para hareketlerinde dağıtık transaction ihtiyacını azaltır; domain sınırlarını korurken atomik commit'i kolaylaştırır.
- Not / trade-off: Microservice kadar bağımsız deploy sağlamaz; fakat v1 için finansal doğruluk ve sadelik daha değerlidir.

Orchestration

- Tanım: Bir use-case içindeki adımların doğru sırada çalıştırılmasıdır.

- FinWallet'ta kullanım: Transfer handler replay -> fraud signals -> internal fraud -> external fraud -> durable fraud -> posting sırasını yönetir.
- Neden / çözdüğü problem: External I/O ile DB transaction sınırlarının doğru kurulmasını sağlar.
- Not / trade-off: Orchestrator'ın tüm business kurallarını kendi içine alması yerine kuralları Domain/Store'a dağıtmak gerekir.

Port

- Tanım: Application katmanının dış dünya ile konuşmak için tanımladığı interface/boundary'dir.
- FinWallet'ta kullanım: IBankProvider, IExternalFraudProvider, store interface'leri port örnekleridir.
- Neden / çözdüğü problem: Application'ın hangi yeteneğe ihtiyaç duyduğunu belirtir, teknolojiyi belirtmez.
- Not / trade-off: Port'un implementasyonu Infrastructure'da tutulur.

Projection

- Tanım: Immutable/history verisinden veya business state'ten okuma için türetilmiş current görünümüdür.
- FinWallet'ta kullanım: Wallet AvailableBalance/BlockedBalance current state'i ledger/history ile reconcile edilen bir projection gibi düşünülür.
- Neden / çözdüğü problem: Her request'te tüm ledger'ı sum etmek yerine hızlı current balance sağlar.
- Not / trade-off: Projection source history ile drift edebilir; reconciliation bu nedenle gereklidir.

Separation of Concerns

- Tanım: Farklı sorumlulukların aynı sınıf veya katmanda birbirine karıştırılmaması prensibidir.
- FinWallet'ta kullanım: HTTP mapping Controller'da, orchestration Application'da, kural Domain'de, SQL/HTTP client Infrastructure'dadır.
- Neden / çözdüğü problem: Değişikliklerin etkilediği alanı küçültür ve okunabilirliği artırır.
- Not / trade-off: Aşırı parçalama da maliyetlidir; proje küçük sınırlar tercih eder.

State Machine / Lifecycle

- Tanım: Bir entity/transaction'ın izin verilen durumlarını ve geçişlerini tanımlayan modeldir.
- FinWallet'ta kullanım: FinancialTransaction ve bank movement akışlarında Scheduled/Pending/Completed/Failed gibi durumlar vardır.
- Neden / çözdüğü problem: Geçersiz state transition'ları engeller ve retry/callback davranışını deterministik yapar.
- Not / trade-off: State isimleri API response değil domain yaşam döngüsüdür.

Stateless Service

- Tanım: Process/container memory'sinde kalıcı iş gerçeği tutmayan servis yaklaşımıdır.
- FinWallet'ta kullanım: Gateway, FinWallet.Api ve fake HTTP servisleri replica olarak yeniden oluşturulabilir; durable finansal state MSSQL'dedir.
- Neden / çözdüğü problem: Horizontal scale ve container restart'ını güvenli hale getirir.
- Not / trade-off: Redis/cache geçici state içerebilir ancak finansal truth memory/container'a bağlanmaz.

Topology

- Tanım: Servislerin, network yollarının, veri kaynaklarının ve güven sınırlarının çalışma zamanındaki yerleşimidir.
- FinWallet'ta kullanım: 25-topology.md Gateway, API, fake provider, MSSQL, Redis, worker ve Docker network ilişkilerini gösterir.
- Neden / çözdüğü problem: Kod klasör yapısından farklı olarak gerçek trafik ve dependency yönünü anlamayı sağlar.
- Not / trade-off: Development ve production topology replica/edge bileşenleri açısından farklılaşabilir.

Transaction Boundary

- Tanım: Bir database transaction'ın nerede başlayıp nerede bittiğini belirleyen tasarım kararıdır.
- FinWallet'ta kullanım: External HTTP çağrısı sırasında SQL transaction açık tutulmaz; provider sonucu alındıktan sonra kısa transaction açılır.
- Neden / çözdüğü problem: Lock süresini ve deadlock riskini azaltır, connection pool'u korur.
- Not / trade-off: Distributed atomicity sağlamaz; dış sistem farkları reconciliation ile ele alınır.

Trust Boundary

- Tanım: Bir taraftan gelen verinin/kimliğin otomatik güvenilir kabul edilmediği güvenlik sınırıdır.
- FinWallet'ta kullanım: Client->Gateway, Gateway->FinWallet.Api, FinWallet.Api->Gateway provider route ve Gateway->provider ayrı trust boundary'lerdir.
- Neden / çözdüğü problem: Gateway bypass veya credential reuse gibi riskleri azaltır.
- Not / trade-off: Her boundary kendi doğrulamasını yapmalıdır.

Use Case

- Tanım: Kullanıcının veya sistemin tek bir iş amacını gerçekleştiren uygulama davranışdır.
- FinWallet'ta kullanım: Register, Login, CreateWallet, BankDeposit, Transfer, Purchase, Refund, Reconciliation birer use-case'tir.
- Neden / çözdüğü problem: Controller ile domain/persistence detayları arasındaki orchestration'ı tek yerde toplar.
- Not / trade-off: Use-case business transaction ile HTTP endpoint birebir aynı olmak zorunda değildir.

Value Object

- Tanım: Kimlikten çok değeri ve geçerlilik kuralları önemli olan immutable domain nesnesidir.
- FinWallet'ta kullanım: Money ve currency gibi kavramlar bu amaçla modellenir.
- Neden / çözdüğü problem: Amount/currency birlikteliği ve geçersiz para değerlerinin domain'e girmemesini sağlar.
- Not / trade-off: Value Object'lar küçük ve immutable tutulmalıdır.

2. API, Gateway ve Ağ (32 terim)

Active Health Check

- Tanım: Proxy'nin belirli aralıklarla backend'e probe göndermesidir.
- FinWallet'ta kullanım: YARP bazı FinWallet/FakeBank/FakeFraud destination'larını aktif olarak kontrol edebilir.
- Neden / çözdüğü problem: Trafik gelmese bile arızayı tespit eder.

- Not / trade-off: Çok sık probe gereksiz yük üretir; interval/timeout config'ten yönetilir.

API (Application Programming Interface)

- Tanım: Bir yazılımın başka yazılımlar tarafından hangi sözleşmeyle çağrılacağını tanımlayan arayüzdür.
- FinWallet'ta kullanım: FinWallet public müşteri API'leri ve internal provider API'leri HTTP üzerinden exposed edilir.
- Neden / çözdüğü problem: Client ile business logic arasında stabil bir sözleşme sağlar.
- Not / trade-off: API sözleşmesi domain modelinin birebir kopyası olmamalıdır; DTO ve ServiceResult ile sınır korunur.

API Gateway

- Tanım: Birden fazla backend'e ortak giriş noktası sağlayan edge uygulamasıdır.
- FinWallet'ta kullanım: FinWallet.Gateway JWT validation, routing, rate-limit, service-key policy ve load balancing uygular.
- Neden / çözdüğü problem: Client'ın backend topology'sini bilmesini engeller ve ortak edge policy'leri merkezi hale getirir.
- Not / trade-off: Gateway içine domain business logic taşınmaz.

Cluster

- Tanım: YARP'ta aynı logical backend servisin bir veya daha fazla destination grubudur.
- FinWallet'ta kullanım: FinWallet.Api ve fake provider'ların her biri kendi cluster'ında tanımlanır.
- Neden / çözdüğü problem: Replica eklenmesini ve load-balancing/health policy uygulanmasını kolaylaştırır.
- Not / trade-off: Development'da tek destination olması cluster kavramını gereksiz yapmaz; production scaling için hazırdır.

Connection Keep-Alive

- Tanım: Aynı TCP/HTTP bağlantısının birden fazla request için yeniden kullanılmasıdır.
- FinWallet'ta kullanım: Kestrel ve outbound HttpClient timeout/lifetime ayarlarıyla kontrollü tutulur.
- Neden / çözdüğü problem: Handshake maliyetini ve latency'yi düşürür.
- Not / trade-off: Aşırı uzun lifetime stale connection/DNS sorunlarına neden olabilir.

Content-Type

- Tanım: HTTP body'nin veri formatını belirten header'dır.
- FinWallet'ta kullanım: Write request'lerinde application/json zorunlu tutulur.
- Neden / çözdüğü problem: Yanlış/ambiguous payload parsing riskini azaltır.
- Not / trade-off: Body olmayan request'lerde JSON zorunluluğu uygulanmaz.

Controller

- Tanım: ASP.NET Core'da HTTP request'i alan ve Application use-case'ine yönlendiren sınıftır.
- FinWallet'ta kullanım: FinWallet Minimal API yerine controller-based API kullanır.
- Neden / çözdüğü problem: HTTP concerns ile business orchestration'ı ayırır ve Swagger metadata'sını düzenli tutar.
- Not / trade-off: Controller kalınlaşırsa business logic katman sızıntısı oluşur.

Correlation ID

- Tanım: Bir request'in servisler arası izini takip etmek için taşınan benzersiz referanstır.
- FinWallet'ta kullanım: X-Correlation-Id Gateway, API ve provider çağrılarında taşınır; geçersizse yeniden üretilir.
- Neden / çözdüğü problem: Dağıtık loglarda aynı iş akışının parçalarını ilişkilendirir.
- Not / trade-off: Idempotency key veya FinancialTransactionId değildir; finansal tekrar güvenliği sağlamaz.

CORS

- Tanım: Browser'ın farklı origin'e yaptığı request'leri hangi origin/method/header kombinasyonunda kabul edeceğini belirleyen politikadır.
- FinWallet'ta kullanım: Allowed origins appsettings üzerinden allow-list olarak yönetilir.
- Neden / çözdüğü problem: Browser tabanlı cross-origin erişimi sınırlar.
- Not / trade-off: CORS authentication değildir ve non-browser client'ları engellemez.

Destination

- Tanım: YARP cluster içindeki tek backend instance adresidir.
- FinWallet'ta kullanım: Örneğin Docker DNS üzerinden http://finwallet-api:8080 bir destination olabilir.
- Neden / çözdüğü problem: Load balancer'ın seçebileceği concrete servis instance'ını temsil eder.
- Not / trade-off: Health check sonucu unhealthy destination trafik dışına alınabilir.

DTO (Data Transfer Object)

- Tanım: Katmanlar veya sistemler arasında veri taşımak için kullanılan sözleşme nesnesidir.
- FinWallet'ta kullanım: Request/response modelleri ile FakeBank/FakeFraud provider contract'ları DTO'dur.
- Neden / çözdüğü problem: Domain entity'lerini doğrudan expose etmeyi engeller.
- Not / trade-off: DTO validation ve mapping maliyeti vardır ama boundary netliği sağlar.

Endpoint

- Tanım: Belirli bir HTTP method + URL kombinasyonu ile erişilen API operasyonudur.
- FinWallet'ta kullanım: POST /api/v1/transfers veya GET /api/v1/transactions endpoint örnekleridir.
- Neden / çözdüğü problem: Use-case'in dış dünyaya açılan giriş noktasıdır.
- Not / trade-off: Endpoint business logic'i içermez; request mapping ve response üretimi yapar.

Fixed-Window Rate Limiter

- Tanım: Zamanı sabit pencerelere bölerek her pencere için izin sayısı uygulayan algoritmadır.
- FinWallet'ta kullanım: Shared.Web global rate limiter bu modeli kullanır.
- Neden / çözdüğü problem: Basit, düşük maliyetli ve config-driven'dir.
- Not / trade-off: Pencere sınırında burst oluşabilir; daha ileri ihtiyaçta sliding/token-bucket düşünülebilir.

Health Check

- Tanım: Bir servisin trafik almaya uygun olup olmadığını kontrol eden mekanizmadır.

- FinWallet'ta kullanım: Gateway/Compose servisleri live/health endpoint'leri ve provider health kontrolleri kullanır.
- Neden / çözdüğü problem: Bozuk instance'a trafik göndermeyi azaltır ve container startup sırasını doğrular.
- Not / trade-off: Health endpoint sadece process ayakta mı değil, gerektiğinde dependency readiness'i de ayırmalıdır.

HTTP Status Code

- Tanım: HTTP response'un genel sonucunu standart numarayla ifade eder.
- FinWallet'ta kullanım: 200, 202, 400, 401, 403, 404, 409, 422, 429 ve 503 FinWallet akışlarında kullanılır.
- Neden / çözdüğü problem: Transport seviyesinde sonucu standartlaştırır; detay machine code ile tamamlanır.
- Not / trade-off: Aynı HTTP status farklı business code'lar taşıyabilir; client sadece status'a bakmamalıdır.

HttpClientFactory

- Tanım: .NET'te HttpClient yaşam döngüsünü ve handler pooling'i merkezi yöneten altyapıdır.
- FinWallet'ta kullanım: Bank, Fraud, Cutoff, Campaign ve Communication typed client'ları bununla oluşturulur.
- Neden / çözdüğü problem: Socket exhaustion ve yanlış HttpClient lifetime kullanımını azaltır.
- Not / trade-off: Financial retry policy otomatik eklenmez; retry business semantics'e göre ayrıca tasarlanır.

JSON

- Tanım: HTTP API'lerde yaygın kullanılan metin tabanlı veri formatıdır.
- FinWallet'ta kullanım: FinWallet write request'lerinde application/json zorunlu tutulur.
- Neden / çözdüğü problem: Platformlar arası kolay serialization ve debugging sağlar.
- Not / trade-off: Finansal decimal ve tarih formatları sözleşmede kontrollü kullanılmalıdır.

Kestrel

- Tanım: ASP.NET Core uygulamalarının HTTP server'ıdır.
- FinWallet'ta kullanım: Shared.Web request body/header, connection, keep-alive ve header timeout sınırlarını Kestrel üzerinden ayarlar.
- Neden / çözdüğü problem: Resource consumption ve header/body abuse risklerini uygulama seviyesinde sınırlar.
- Not / trade-off: Internet edge protection yerine geçmez; reverse proxy/LB arkasında çalışabilir.

Load Balancing

- Tanım: İsteklerin birden fazla backend instance arasında dağıtılmasıdır.
- FinWallet'ta kullanım: YARP production'da aynı cluster'a eklenen replica'lar arasında trafik dağıtılabilir.
- Neden / çözdüğü problem: Kapasite ve availability artırır.
- Not / trade-off: Stateful correctness load balancer sticky-session'a bağlı olmamalıdır; durable state MSSQL'de olmalıdır.

OpenAPI

- Tanım: REST API sözleşmesini makine tarafından okunabilir formatta tanımlayan spesifikasyondur.
- FinWallet'ta kullanım: Swagger tooling arka planda OpenAPI document üretir; kullanıcıya yönelik terim olarak Swagger daha çok görünür.
- Neden / çözdüğü problem: Tooling/client generation ve schema keşfi sağlar.
- Not / trade-off: FinWallet domain mimarisi OpenAPI'ye bağımlı değildir.

Passive Health Check

- Tanım: Gerçek trafik sırasında oluşan hatalara göre destination health değerlendirmesidir.
- FinWallet'ta kullanım: FinWallet API cluster'ında transport failure sonrası destination geçici olarak devre dışı bırakılabilir.
- Neden / çözdüğü problem: Ek probe olmadan gerçek failure sinyali kullanır.
- Not / trade-off: Düşük trafik ortamında arızayı geç fark edebilir; active check ile birlikte daha güçlüdür.

PowerOfTwoChoices

- Tanım: Load balancing'de rastgele iki candidate seçip daha az yüklü olana yönlendiren algoritmadır.
- FinWallet'ta kullanım: FinWallet YARP cluster'larında tercih edilen policy'dir.
- Neden / çözdüğü problem: Round-robin'e göre çok az ek maliyetle daha iyi yük dağılımı sağlayabilir.
- Not / trade-off: Tek destination olduğunda fiilen seçim yapmaz; replica sayısı arttığında anlam kazanır.

Rate Limiting

- Tanım: Belirli süre içinde kabul edilen istek sayısını sınırlamaktır.
- FinWallet'ta kullanım: Gateway per-IP fixed-window limit uygular; backend de defense-in-depth limiti tutabilir.
- Neden / çözdüğü problem: Resource exhaustion, brute-force ve basit L7 abuse etkisini azaltır.
- Not / trade-off: Volumetric DDoS çözümü değildir; edge DDoS/WAF yine gerekir.

REST

- Tanım: HTTP resource/verb semantiğini kullanan web API yaklaşımıdır.
- FinWallet'ta kullanım: Controller tabanlı endpoint'lerde GET/POST gibi HTTP methodları ve JSON payload'lar kullanılır.
- Neden / çözdüğü problem: Basit, yaygın ve tooling desteği güçlü bir entegrasyon modeli sağlar.
- Not / trade-off: FinWallet tam HATEOAS odaklı akademik REST uygulamaz; pragmatik REST API kullanır.

Retry-After

- Tanım: Client'a ne kadar süre sonra tekrar denemesi gerektiğini bildiren HTTP header'ıdır.
- FinWallet'ta kullanım: Rate limit rejection'ında mümkünse response'a eklenir.
- Neden / çözdüğü problem: Client'ın hemen tekrar tekrar request atmasını azaltır.
- Not / trade-off: Financial POST'ların retry davranışı ayrıca idempotency ve provider semantics'e bağlıdır.

Reverse Proxy

- Tanım: Client isteğini alıp arka taraftaki başka bir servise ileten ara HTTP bileşenidir.
- FinWallet'ta kullanım: Gateway public /api/* isteklerini FinWallet.Api'ye, /providers/* isteklerini fake provider'lara iletir.
- Neden / çözdüğü problem: Backend adreslerini client'tan gizler ve merkezi trafik kontrolü sağlar.
- Not / trade-off: Proxy business source of truth olmamalıdır; finansal state Gateway'de tutulmaz.

Route

- Tanım: Gateway'in gelen URL/path'i hangi backend cluster'a göndereceğini belirleyen kuraldır.
- FinWallet'ta kullanım: Public auth, protected API, internal callback ve /providers/* için ayrı YARP route'ları vardır.
- Neden / çözdüğü problem: Authentication policy ve routing önceliğini path seviyesinde kontrol eder.
- Not / trade-off: Route sırası/precedence yanlışsa internal endpoint yanlış auth policy'ye düşebilir.

ServiceResult<T>

- Tanım: FinWallet'ın success/failure response'larını tek biçimde taşıyan ortak envelope modelidir.
- FinWallet'ta kullanım: isSuccess, code, message, data, errors alanlarıyla tüm API'lerde tutarlı response üretilir.
- Neden / çözdüğü problem: Client'ın her endpoint için farklı hata şekli parse etmesini engeller.
- Not / trade-off: HTTP status'un yerini almaz; HTTP status + ServiceResult code birlikte kullanılır.

SocketsHttpClient

- Tanım: .NET HttpClient'in connection pooling, DNS/lifetime ve transport davranışını yöneten low-level handler'dır.
- FinWallet'ta kullanım: PooledConnectionLifetime, idle timeout ve MaxConnectionsPerServer gibi ayarlar config'ten verilir.
- Neden / çözdüğü problem: Uzun yaşayan servislerde connection reuse ve DNS yenilenmesini dengeler.
- Not / trade-off: Değerler ölçüm olmadan aşırı yükseltilmemelidir.

Swagger

- Tanım: API endpoint, request/response ve schema'ları interaktif olarak keşfetmeyi sağlayan dokümantasyon arayüzüdür.
- FinWallet'ta kullanım: Gateway, FinWallet.Api ve tüm fake API'lerde Shared.Web/Swashbuckle ile bulunur.
- Neden / çözdüğü problem: Developer onboarding, manual test ve contract görünürlüğünü kolaylaştırır.
- Not / trade-off: Production'da varsayılan kapalıdır; açık olması endpoint authorization'ını bypass etmez.

Timeout

- Tanım: Bir I/O operasyonunun sonsuza kadar beklemesini engelleyen süre sınırıdır.
- FinWallet'ta kullanım: Provider HttpClient, YARP activity ve request header süreleri config'ten sınırlandırılır.
- Neden / çözdüğü problem: Thread/connection kaynaklarının kilitletmesini ve cascading stall riskini azaltır.
- Not / trade-off: Çok kısa timeout false failure; çok uzun timeout resource exhaustion üretir.

YARP

- Tanım: Microsoft'un .NET için reverse-proxy toolkit'idir.
- FinWallet'ta kullanım: FinWallet.Gateway route, cluster, load balancing, health ve request transform davranışlarını YARP ile uygular.
- Neden / çözdüğü problem: Gateway'i özel low-level proxy kodu yazmadan config-driven yönetmeyi sağlar.
- Not / trade-off: YARP uygulama gateway'idir; volumetric DDoS için tek başına edge/WAF yerine geçmez.

3. Güvenlik ve Kimlik (39 terim)

sid Claim

- Tanım: JWT içindeki session kimliğini taşıyan claim'dir.
- FinWallet'ta kullanım: Login'da oluşturulan durable session ID access token'a yazılır ve logout/fraud signal'larında kullanılır.
- Neden / çözdüğü problem: Token'ı aktif session state'yle ilişkilendirir ve revoke kontrolü yapılmasını sağlar.
- Not / trade-off: Sadece token signature'a güvenmek yerine session lifecycle doğrulaması ekler.

sub Claim

- Tanım: JWT standardında subject/ana kimlik claim'idir.
- FinWallet'ta kullanım: FinWallet'ta authenticated CustomerId'yi taşır.
- Neden / çözdüğü problem: Controller'ın request body'den müşteri kimliği kabul etmeden owner context üretmesini sağlar.
- Not / trade-off: Geçersiz/eksik GUID ise request INVALID_ACCESS_TOKEN ile reddedilir.

Access Token

- Tanım: Kısa ömürlü, protected API çağrılarında kullanılan credential'dır.
- FinWallet'ta kullanım: Login/refresh sonrası JWT olarak üretilir.
- Neden / çözdüğü problem: Her request'te kullanıcı kimliğini doğrular.
- Not / trade-off: Kısa lifetime çalınmış token riskini azaltır ama refresh mekanizması gerektirir.

Authentication

- Tanım: Bir isteği yapan kişinin veya servisin kim olduğunu doğrulama işlemidir.
- FinWallet'ta kullanım: Public client için JWT/login; internal servisler için service-key kullanılır.
- Neden / çözdüğü problem: Kimliği doğrulanmamış aktörün korumalı endpoint'e erişmesini engeller.
- Not / trade-off: Authentication kimliğin ne yapabileceğini belirlemez; authorization ayrı kontroldür.

Authorization

- Tanım: Doğrulanmış kimliğin belirli bir kaynağa/işleme izinli olup olmadığını kontrol etmektir.
- FinWallet'ta kullanım: Gateway route policy'leri, [Authorize], owner-aware SQL ve internal service policy'leri kullanılır.
- Neden / çözdüğü problem: Başka müşterinin wallet/transaction verisine erişim gibi BOLA risklerini azaltır.

- Not / trade-off: Sadece role bakmak yeterli değildir; resource ownership de doğrulanır.

Bearer Token

- Tanım: Token'a sahip olan tarafın onu HTTP Authorization header ile sunduğu access-token kullanım biçimidir.
- FinWallet'ta kullanım: Authorization: Bearer <JWT> protected client endpoint'lerinde kullanılır.
- Neden / çözdüğü problem: Standart HTTP tooling ile kolay entegrasyon sağlar.
- Not / trade-off: Token ele geçirilirse süresi dolana veya session revoke edilene kadar risk oluşturur; bu yüzden TLS ve kısa lifetime gerekir.

BOLA (Broken Object Level Authorization)

- Tanım: Authenticated kullanıcının başka kullanıcıya ait object'i ID değiştirerek erişebilmesi zafiyetidir.
- FinWallet'ta kullanım: Wallet, transaction ve bank-account sorgularında customer ownership server-side doğrulanır.
- Neden / çözdüğü problem: ID tahmini veya request manipulation ile başka müşterinin verisine erişimi engeller.
- Not / trade-off: Sadece JWT doğrulamak BOLA'yı çözmez; object-level owner check şarttır.

CDN

- Tanım: İçeriği edge noktalarında cache/serve eden dağıtık delivery network'üdür.
- FinWallet'ta kullanım: FinWallet API için zorunlu component değildir; DDoS/edge architecture tartışmalarında olası production katmanı olarak geçer.
- Neden / çözdüğü problem: Static content ve edge traffic absorption sağlar.
- Not / trade-off: Authenticated financial API request'leri için caching politikaları çok dikkatli yönetilmelidir.

Claim

- Tanım: JWT içinde kimlik veya session hakkında taşınan isim/değer bilgisidir.
- FinWallet'ta kullanım: sub customer identity, sid session identity için kullanılır.
- Neden / çözdüğü problem: Gateway/API'nin request'i server-side identity ile ilişkilendirmesini sağlar.
- Not / trade-off: Client body'den gelen customerId yerine token claim'leri tercih edilerek spoofing azaltılır.

CSP (Content Security Policy)

- Tanım: Browser'ın hangi script/style/frame kaynaklarını çalıştırabileceğini sınırlayan response policy'sidir.
- FinWallet'ta kullanım: Shared.Web API response'lerinde restrictive CSP; Swagger için daha uygun ayrı CSP kullanır.
- Neden / çözdüğü problem: XSS/frame injection etkisini azaltan browser defense-in-depth katmanıdır.
- Not / trade-off: API'nin kendisi UI olmadığı için etkisi sınırlı ama güvenli default sağlar.

Data Masking

- Tanım: Log veya response'ta hassas verinin tamamını göstermeyip güvenli şekilde kısmen gizlemektir.
- FinWallet'ta kullanım: Token, OTP, password ve secret hiç loglanmaz; PII gerektiğinde minimize/mask edilir.

- Neden / çözdüğü problem: Operational görünürlük ile privacy/security arasında denge sağlar.
- Not / trade-off: Masking encryption değildir; masked veri kaynak sistemde hâlâ korunmalıdır.

DDoS Protection

- Tanım: Çok yüksek hacimli dağıtık trafik saldırılarında network/edge kapasitesini koruyan altyapı mekanizmasıdır.
- FinWallet'ta kullanım: App-level rate limits yalnız L7 abuse sınırlar; production'da cloud/LB/CDN DDoS koruması gerekir.
- Neden / çözdüğü problem: Volumetric trafik uygulamaya ulaşmadan absorbe/filter edilir.
- Not / trade-off: Kestrel rate limit tek başına DDoS çözümü değildir.

DownstreamServiceKey

- Tanım: Gateway'in FinWallet.Api veya fake provider'a proxy ettiği isteğin Gateway'den geldiğini kanıtlayan ayrı secret'tir.
- FinWallet'ta kullanım: YARP transform proxied request'e header ekler; destination Shared.Web kontrol eder.
- Neden / çözdüğü problem: Gateway bypass edilip backend portuna doğrudan erişildiğinde business endpoint'in trusted olmamasını sağlar.
- Not / trade-off: InternalServiceKey ile aynı secret kullanılmaz; trust boundary ayrımı korunur.

Fail-Closed

- Tanım: Güvenlik/karar servisi hata verdiğinde işlemi güvenli tarafta reddetme yaklaşımıdır.
- FinWallet'ta kullanım: External fraud provider unavailable olduğunda protected transfer/purchase devam etmez ve 503 döner.
- Neden / çözdüğü problem: Risk değerlendirmesi olmadan para hareketinin gerçekleşmesini engeller.
- Not / trade-off: Availability düşebilir; hangi dependency'nin fail-closed olması business risk iştahına bağlıdır.

Fail-Open

- Tanım: Kontrol servisi hata verdiğinde işlemin yine de devam etmesi yaklaşımıdır.
- FinWallet'ta kullanım: FinWallet fraud kararında bilinçli olarak kullanılmaz; communication gibi post-commit yan etkilerde ise para rollback edilmez.
- Neden / çözdüğü problem: Bazı non-critical dependency'lerde availability'yi koruyabilir.
- Not / trade-off: Fraud/auth gibi kritik kararlarda finansal risk yaratır; bu yüzden kullanım alanı dikkatle ayrılır.

Fixed-Time Comparison

- Tanım: Secret karşılaştırmalarında karakter karakter erken çıkış yapmayarak timing side-channel riskini azaltan karşılaştırmadır.
- FinWallet'ta kullanım: Internal service key doğrulamasında CryptographicOperations.FixedTimeEquals kullanılır.
- Neden / çözdüğü problem: Doğru secret'in prefix bilgisinin response süresinden tahmin edilmesini zorlaştırır.
- Not / trade-off: Önce uzunluk kontrolü yine gerekir; tamamen tüm side-channel'ları çözmez.

HMAC

- Tanım: Bir secret key ile mesaj bütünlüğü/doğrulama değeri üretme mekanizmasıdır.
- FinWallet'ta kullanım: JWT HS256 ve OTP doğrulama gibi alanlarda HMAC yaklaşımı kullanılır.
- Neden / çözdüğü problem: Mesajın secret'i bilmeyen biri tarafından geçerli biçimde üretilmemesini sağlar.
- Not / trade-off: Encryption değildir; veriyi gizlemez, doğruluk/authenticity sağlar.

HS256 / HMAC-SHA256

- Tanım: JWT imzasında paylaşılan secret ile SHA-256 tabanlı HMAC kullanan simetrik algoritmadır.
- FinWallet'ta kullanım: FinWallet access token'larında fixed algorithm olarak kullanılır; signing key en az 32 UTF-8 byte olmalıdır.
- Neden / çözdüğü problem: Basit ve hızlıdır; tek güven alanında yeterli olabilir.
- Not / trade-off: Secret'i bilen herkes token imzalayabilir; çoklu trust domain için asymmetric key düşünülebilir.

HSTS

- Tanım: Browser'a belirli süre boyunca siteye sadece HTTPS ile bağlanmasını söyleyen response policy'sidir.
- FinWallet'ta kullanım: Production'da config ile etkinleştirilebilir.
- Neden / çözdüğü problem: HTTPS downgrade/accidental HTTP kullanımını azaltır.
- Not / trade-off: Local development ve TLS termination topology'si dikkate alınarak kullanılmalıdır.

InternalServiceKey

- Tanım: FinWallet.Api'nin Gateway'in internal provider route'larına trusted caller olduğunu kanıtlayan secret'tir.
- FinWallet'ta kullanım: X-Internal-Service-Key ile /providers/* veya internal route'a gönderilir ve Gateway policy doğrular.
- Neden / çözdüğü problem: Normal client'in provider route'larını doğrudan kullanmasını engeller.
- Not / trade-off: Production'da secret-store/env injection ile verilmeli ve yeterince uzun olmalıdır.

JWT (JSON Web Token)

- Tanım: İmzalı claim'ler taşıyan stateless access-token formatıdır.
- FinWallet'ta kullanım: Login sonrası access token üretilir; Gateway ve FinWallet.Api issuer/audience/signature/lifetime doğrular.
- Neden / çözdüğü problem: Her request'te DB'den kimlik bilgisi taşımadan authenticated identity sağlar.
- Not / trade-off: JWT iptali doğal olarak zor olduğundan sid session state'yle birlikte kullanılır.

Least Privilege

- Tanım: Kullanıcı, servis veya workflow'a yalnız gerekli minimum yetkinin verilmesi güvenlik prensibidir.
- FinWallet'ta kullanım: Gateway internal route'ları ayrı key/policy, GitHub workflow permissions sınırlı ve backend portları public değil tutulur.
- Neden / çözdüğü problem: Credential compromise durumunda blast radius'u azaltır.
- Not / trade-off: Gereğinden az yetki operasyonu kırabilir; yetki ihtiyaçları açıkça belgelenmelidir.

Logout / Session Revocation

- Tanım: Aktif session'ın server-side geçersiz hale getirilmesidir.
- FinWallet'ta kullanım: POST /api/v1/auth/logout JWT içindeki sid session'ını revoke eder.
- Neden / çözdüğü problem: Access token henüz expire olmamış olsa bile API'nin session kontrolüyle kullanımı engellenebilir.
- Not / trade-off: Tam etki API'nin her korumalı akışta session doğrulaması yapmasına bağlıdır.

OTP (One-Time Password)

- Tanım: Tek kullanımlık ve kısa ömürlü doğrulama kodudur.
- FinWallet'ta kullanım: Registration verification için SMS üzerinden gönderilir; TTL'li geçici state kullanılır.
- Neden / çözdüğü problem: Telefon sahipliği gibi ikinci bir doğrulama adımı sağlar.
- Not / trade-off: OTP access token değildir; log/response içinde plaintext sızdırılmamalıdır.

OWASP API Security Top 10

- Tanım: API'lere özgü BOLA, broken authentication, resource consumption ve unsafe consumption gibi riskleri sınıflandırır.
- FinWallet'ta kullanım: Gateway/API auth, owner-aware SQL, rate limit ve provider ACL tasarımında referans alınır.
- Neden / çözdüğü problem: API threat'lerini klasik web UI risklerinden daha net ayırır.
- Not / trade-off: Liste tek başına yeterli değildir; business abuse/fraud ayrıca ele alınır.

OWASP Top 10

- Tanım: Web uygulamalarındaki yaygın güvenlik risklerini sınıflandıran OWASP rehberidir.
- FinWallet'ta kullanım: FinWallet security dokümanı access control, misconfiguration, cryptography, injection, logging gibi başlıklarla mapping yapar.
- Neden / çözdüğü problem: Güvenlik review'unda ortak checklist/dil sağlar.
- Not / trade-off: Compliance sertifikası değildir; threat model ve gerçek testlerin yerini almaz.

PBKDF2

- Tanım: Password'ları brute-force'a karşı yavaşlatılmış şekilde hash'lemek için kullanılan key-derivation algoritmasıdır.
- FinWallet'ta kullanım: FinWallet credential storage'da PBKDF2 V1 sabit work factor ve salt ile kullanılır.
- Neden / çözdüğü problem: Plaintext password veya hızlı hash saklama riskini azaltır.
- Not / trade-off: Iteration/work factor config'ten rastgele değiştirilemez; versioned migration/re-hash gerekir.

Pepper

- Tanım: Hash/HMAC işlemlerine uygulama secret'ı olarak eklenen, veritabanından ayrı tutulan gizli değerdir.
- FinWallet'ta kullanım: Registration OTP doğrulamasında REGISTRATION_OTP_PEPPER gibi secret kullanılabilir.
- Neden / çözdüğü problem: Sadece DB ele geçirilse bile doğrulama material'ının tek başına yeterli olmamasını sağlar.

- Not / trade-off: Pepper rotate edilmesi operasyonel plan gerektirir; source control'a konmaz.

PII (Personally Identifiable Information)

- Tanım: Bir gerçek kişiyi doğrudan veya dolaylı tanımlayabilen veridir.
- FinWallet'ta kullanım: Phone/email gibi registration verileri business için tutulur; fraud/reconciliation response'larında gereksiz PII çıkarılmaz.
- Neden / çözdüğü problem: Log ve internal API'lerde veri minimizasyonu sağlar.
- Not / trade-off: PII tanımı regülasyona göre değişebilir; production privacy sınıflandırması gerekir.

Refresh Token

- Tanım: Access token süresi dolduğunda yeni access token almak için kullanılan daha uzun ömürlü opaque credential'dır.
- FinWallet'ta kullanım: FinWallet refresh session akışında durable state ve rotation ile yönetilir.
- Neden / çözdüğü problem: Kullanıcıyı sık login ettirmeden access token'ı kısa tutmayı sağlar.
- Not / trade-off: Çalınması yüksek risklidir; hash/rotation/revocation ve güvenli saklama gerekir.

Refresh Token Rotation

- Tanım: Her başarılı refresh'te eski refresh token'ın geçersizleşip yenisinin üretilmesidir.
- FinWallet'ta kullanım: FinWallet session güvenliğinde replay riskini azaltmak için kullanılır.
- Neden / çözdüğü problem: Ele geçirilen eski refresh token'ın tekrar kullanımını sınırlar.
- Not / trade-off: Concurrent refresh race'leri durable transaction/unique state ile test edilmelidir.

Salt

- Tanım: Password hash'e kullanıcı/credential başına eklenen rastgele değerdir.
- FinWallet'ta kullanım: PBKDF2 hash kayıtları benzersiz salt kullanır.
- Neden / çözdüğü problem: Aynı password'ün aynı hash'i üretmesini ve precomputed rainbow-table etkisini azaltır.
- Not / trade-off: Salt secret olmak zorunda değildir; her credential için benzersiz ve yeterince rastgele olmalıdır.

Secret Injection

- Tanım: Signing key, DB password veya service key gibi secret'ların source code yerine deployment ortamından verilmesidir.
- FinWallet'ta kullanım: Production appsettings placeholder bırakır; environment/secret-store override beklenir.
- Neden / çözdüğü problem: Secret'ın Git history veya image içine gömülmesini önler.
- Not / trade-off: Local .env yalnız development kolaylığıdır; production secret manager'ın alternatifi değildir.

Security Headers

- Tanım: Browser/client davranışını daha güvenli default'lara yönlendiren HTTP response header setidir.
- FinWallet'ta kullanım: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, CSP ve no-store gibi header'lar Shared.Web'de merkezi uygulanır.
- Neden / çözdüğü problem: Tek tek endpoint'lerin güvenlik header'ı unutmasını önler.

- Not / trade-off: Authentication/authorization yerine geçmez; ek hardening katmanıdır.

Service-to-Service Authentication

- Tanım: İnsan kullanıcı yerine bir backend servisin başka servise kimliğini kanıtlamasıdır.
- FinWallet'ta kullanım: FinWallet.Api -> Gateway için InternalServiceKey, Gateway -> backend/provider için DownstreamServiceKey kullanılır.
- Neden / çözdüğü problem: Public JWT'nin internal machine trust için yeniden kullanılmasını engeller.
- Not / trade-off: Static key yerine production'da mTLS/managed identity gibi daha güçlü yöntemler düşünülebilir.

Session

- Tanım: Authenticated kullanıcının login yaşam döngüsünü temsil eden server-side kayıttır.
- FinWallet'ta kullanım: sid, device ve revoke/refresh bilgileri MSSQL'de durable tutulur.
- Neden / çözdüğü problem: JWT stateless olsa bile logout, revocation ve risk sinyali için server-side kontrol sağlar.
- Not / trade-off: Session state financial source of truth değildir ama auth correctness için authoritative'dir.

SQL Injection

- Tanım: User input'ının SQL komut yapısına karışarak beklenmeyen query çalıştırması zafiyetidir.
- FinWallet'ta kullanım: FinWallet SQL erişiminde parameterized command kullanır ve user-controlled SQL fragment kabul etmez.
- Neden / çözdüğü problem: Veri sızması/değiştirme ve privilege abuse riskini azaltır.
- Not / trade-off: Parametre kullanmak dynamic table/column isimlerini otomatik güvenli yapmaz; bunlar server-owned olmalıdır.

SSRF (Server-Side Request Forgery)

- Tanım: Client input'ıyla server'ın istenmeyen internal/external URL'lere request atmaya zorlanması zafiyetidir.
- FinWallet'ta kullanım: Provider base URL'leri server-owned configuration'dır; client arbitrary URL gönderemez.
- Neden / çözdüğü problem: Metadata service/internal network erişimi gibi riskleri azaltır.
- Not / trade-off: Config/secrets ele geçirilirse ayrı risk vardır; network egress policy yine faydalıdır.

WAF (Web Application Firewall)

- Tanım: HTTP/L7 saldırı pattern'lerini edge'de filtreleyen güvenlik katmanıdır.
- FinWallet'ta kullanım: FinWallet code içinde WAF yoktur; production DDoS/OWASP edge protection için dış altyapı olarak önerilir.
- Neden / çözdüğü problem: Uygulama daha request'i almadan bazı abuse/signature trafiğini azaltır.
- Not / trade-off: Business authorization/fraud kurallarının alternatifi değildir.

4. Veri, Persistence ve Concurrency (36 terim)

ACID

- Tanım: Database transaction'larının Atomicity, Consistency, Isolation ve Durability özelliklerini özetleyen kavramdır.
- FinWallet'ta kullanım: Financial posting MSSQL transaction'ları bu özelliklerden yararlanır.
- Neden / çözdüğü problem: Para hareketinin yarım commit edilmemesi ve concurrent işlemlerin kuralları bozmaması için temel sağlar.
- Not / trade-off: External HTTP ACID transaction'ın parçası değildir; distributed consistency ayrıca yönetilir.

Atomicity

- Tanım: Bir transaction içindeki değişikliklerin ya tamamen uygulanması ya da hiç uygulanmamasıdır.
- FinWallet'ta kullanım: Transfer'da source debit, destination credit, FinancialTransaction, Ledger ve Idempotency birlikte commit/rollback olur.
- Neden / çözdüğü problem: Tek taraflı para hareketini engeller.
- Not / trade-off: External provider çağrısı bu atomicity'nin dışında olduğundan compensation/reconciliation gerekir.

Check Constraint

- Tanım: Bir row değerinin belirli mantıksal koşulu sağlamasını DB seviyesinde zorunlu tutar.
- FinWallet'ta kullanım: Amount/status/type/financial invariant'ların bazıları schema seviyesinde desteklenir.
- Neden / çözdüğü problem: Application bug'ı olsa bile invalid row yazılmasını azaltır.
- Not / trade-off: Komple business flow'u DB constraint'e taşımak yerine basit invariant'lar için kullanılır.

Commit

- Tanım: Database transaction'ındaki değişiklikleri kalıcı hale getirmektir.
- FinWallet'ta kullanım: Finansal endpoint başarıya gitmeden önce ilgili atomic posting commit edilir.
- Neden / çözdüğü problem: Success response ile durable state arasındaki sıralamayı netleştirir.
- Not / trade-off: Commit sonrası communication failure para transaction'ını geri alamaz; outbox retry devreye girer.

Connection Pooling

- Tanım: DB veya HTTP connection'larının her işlemde fiziksel olarak yeniden açılmak yerine havuzdan reuse edilmesidir.
- FinWallet'ta kullanım: SqlConnection logical open/close provider pool kullanır; HttpClient da handler connection pool kullanır.
- Neden / çözdüğü problem: Handshake/login maliyetini ve latency'yi düşürür.
- Not / trade-off: Pool size yanlış ayarlanırsa DB'yi boğabilir veya request'leri kuyrukta bekletebilir.

Consistency (Database)

- Tanım: Transaction öncesi ve sonrası verinin constraint/invariant'lara uygun geçerli durumda kalmasıdır.
- FinWallet'ta kullanım: Ledger balance, non-negative wallet ve foreign/unique constraints bu amacı destekler.
- Neden / çözdüğü problem: DB'nin invalid state kabul etmesini azaltır.

- Not / trade-off: Business consistency sadece DB constraint ile değil Application/Domain kurallarıyla birlikte sağlanır.

Database Migration

- Tanım: Schema'nın bir versiyondan diğerine kontrollü değiştirilmesidir.
- FinWallet'ta kullanım: Docker mssql-init 001, 002, 003, 004 gibi scriptleri SchemaVersions kontrolüyle sıralı uygular.
- Neden / çözdüğü problem: Yeni kodun ihtiyaç duyduğu schema'yı tekrarlanabilir şekilde kurar.
- Not / trade-off: Migration tekrar çalıştırılabilirlik/idempotency ve rollback/forward-fix planı gerektirir.

Database Transaction

- Tanım: Bir grup SQL değişikliğini tek commit/rollback birimi olarak çalıştıran mekanizmadır.
- FinWallet'ta kullanım: Wallet posting, correction, outbox claim/finalization ve bazı auth state değişimleri transaction içinde yapılır.
- Neden / çözdüğü problem: Birbiriyle ilişkili row'ların birlikte değişmesini sağlar.
- Not / trade-off: External HTTP çağrısı transaction içinde tutulmaz.

Decimal Precision

- Tanım: Parasal değerlerin floating-point yerine sabit ondalık hassasiyetle saklanmasıdır.
- FinWallet'ta kullanım: Money amount alanları decimal/DB decimal olarak tutulur.
- Neden / çözdüğü problem: Binary floating-point rounding hatalarının finansal hesaplara sızmasını engeller.
- Not / trade-off: Scale/rounding kuralları domain invariant olarak tutarlı olmalıdır.

Durability

- Tanım: Commit edilmiş transaction'ın crash sonrası kaybolmaması özelliğidir.
- FinWallet'ta kullanım: Completed transaction ve ledger kayıtları MSSQL commit sonrası kalıcı kabul edilir.
- Neden / çözdüğü problem: Client'a başarı döndükten sonra finansal state'in ortadan kaybolmamasını sağlar.
- Not / trade-off: HA/backup/disaster-recovery durability'nin altyapı boyutunu tamamlar.

Durable State

- Tanım: Process/container restart olsa bile kalması gereken persist edilmiş veridir.
- FinWallet'ta kullanım: Wallet balance, ledger, transaction, idempotency, session, fraud review ve outbox/inbox MSSQL'de durable tutulur.
- Neden / çözdüğü problem: Crash/restart sonrasında correctness ve replay davranışını korur.
- Not / trade-off: Durable olmak tek başına doğru olmak değildir; transaction/constraint kuralları da gerekir.

Filtered Index

- Tanım: Sadece belirli koşulu sağlayan row'ları indexleyen SQL Server index türüdür.
- FinWallet'ta kullanım: Pending/active gibi küçük subset sorgularında schema tarafından kullanılabilir.
- Neden / çözdüğü problem: Daha küçük index ile targeted query performansı sağlar.

- Not / trade-off: SQL Server session SET option'ları (ör. QUOTED_IDENTIFIER) migration sırasında doğru olmalıdır.

Foreign Key

- Tanım: Bir tablodaki referansın başka tablodaki mevcut row'a işaret etmesini zorunlu tutan constraint'tir.
- FinWallet'ta kullanım: Transaction detail, ledger entry ve wallet/customer ilişkileri gibi bağlantıları korur.
- Neden / çözdüğü problem: Orphan data oluşmasını azaltır.
- Not / trade-off: High-write sistemlerde constraint maliyeti vardır ama finansal bütünlük için genellikle değerlidir.

GUID / UUID

- Tanım: Dağıtık ortamlarda benzersiz kimlik üretmek için kullanılan 128-bit identifier formatıdır.
- FinWallet'ta kullanım: Customer, Session, Wallet, Transaction, Journal, FraudEvent gibi birçok entity kimliği GUID'dir.
- Neden / çözdüğü problem: Merkezi sequence koordinasyonu olmadan kimlik üretimini kolaylaştırır.
- Not / trade-off: Random GUID clustered index fragmentation yaratabilir; index tasarımı ayrı düşünülmelidir.

Index

- Tanım: Query'lerin row'lara daha hızlı ulaşmasını sağlayan veri yapısıdır.
- FinWallet'ta kullanım: Customer/transaction/idempotency/history sorgularında uygun key'ler için kullanılır.
- Neden / çözdüğü problem: Logical read ve latency'yi düşürebilir.
- Not / trade-off: Her index write/storage maliyeti getirir; Query Store/plan/ölçüm olmadan rastgele eklenmez.

Isolation

- Tanım: Concurrent transaction'ların birbirlerinin ara state'ini ne ölçüde görebildiğini belirleyen özelliktir.
- FinWallet'ta kullanım: Financial posting store'larında overspend/idempotency race'i önlemek için güçlü isolation/locking kullanılır.
- Neden / çözdüğü problem: Aynı wallet üzerinde eşzamanlı debit'lerin toplam bakiyeyi aşmasını engellemeye yardımcı olur.
- Not / trade-off: Yüksek isolation lock contention yaratabilir; finansal hot-path ölçülerek ayarlanmalıdır.

Keyset Pagination

- Tanım: Son görülen kayıt key'ini cursor olarak kullanıp sonraki sayfayı WHERE < cursor mantığıyla getiren pagination yöntemidir.
- FinWallet'ta kullanım: Transaction history beforeTransactionId ve ordering key'leriyle newest-first okunur.
- Neden / çözdüğü problem: Büyük OFFSET değerlerinde oluşan scan maliyetini azaltır ve concurrent insert'lerde daha stabil sayfalama sağlar.
- Not / trade-off: Arbitrary page number'a doğrudan atlamak zordur; cursor tabanlı UX gerektirir.

Lock

- Tanım: Concurrent işlemlerin aynı veriyi uyumsuz şekilde değiştirmesini engelleyen DB concurrency mekanizmasıdır.
- FinWallet'ta kullanım: Wallet debit ve idempotency create/finalize sırasında SQL locking correctness'in parçasıdır.
- Neden / çözdüğü problem: İki request'in aynı bakiyeyi okuyup ikisinin de harcamasını önler.
- Not / trade-off: Uzun transaction ve yanlış lock order deadlock riskini artırır.

MSSQL / SQL Server

- Tanım: İlişkisel veritabanı yönetim sistemidir.
- FinWallet'ta kullanım: FinWallet'ın Customer, Session, Wallet, BankAccount, FinancialTransaction, Ledger, Idempotency, FraudEvent, Outbox/Inbox ve Reconciliation için durable source of truth'üdür.
- Neden / çözdüğü problem: ACID transaction, constraint, locking ve güçlü query desteği finansal correctness sağlar.
- Not / trade-off: Scale gereksinimi arttığında indexing/partitioning/HA planı gerekir; Redis yerine geçici cache amacıyla kullanılmaz.

Optimistic Concurrency

- Tanım: Çakışmanın nadir olduğu varsayımıyla version/CAS kontrolüyle update sırasında conflict yakalama yaklaşımıdır.
- FinWallet'ta kullanım: FinWallet bazı alanlarda DB uniqueness/compare-and-set semantics kullanabilir; ana wallet posting güçlü locking de kullanır.
- Neden / çözdüğü problem: Lock süresini azaltabilir.
- Not / trade-off: Hot financial balance'da yüksek conflict varsa pessimistic locking daha uygun olabilir.

Parameterized SQL

- Tanım: SQL query değerlerini string concatenation yerine parameter olarak göndermektir.
- FinWallet'ta kullanım: FinWallet store'ları customerId, amount, status vb. değerleri SQL parameters ile kullanır.
- Neden / çözdüğü problem: SQL injection riskini azaltır ve query plan reuse sağlayabilir.
- Not / trade-off: Table/column isimleri parameter olamaz; server-owned kalmalıdır.

Persistence

- Tanım: Uygulama state'inin kalıcı veri store'una yazılması ve okunmasıdır.
- FinWallet'ta kullanım: Infrastructure SQL store'ları domain/application state'ini MSSQL'e persist eder.
- Neden / çözdüğü problem: Process memory'den bağımsız yaşam döngüsü sağlar.
- Not / trade-off: Persistence modeli domain'i SQL tablo yapısına bağımlı hale getirmemelidir.

Pessimistic Concurrency

- Tanım: Çakışma ihtimalini yüksek kabul edip işlem sırasında ilgili veriyi lock'lama yaklaşımıdır.
- FinWallet'ta kullanım: Critical wallet balance posting'inde aynı row'un concurrent debit'leri kontrol edilir.
- Neden / çözdüğü problem: Overspend riskini doğrudan DB transaction içinde önler.
- Not / trade-off: Throughput'u lock contention nedeniyle sınırlayabilir.

Race Condition

- Tanım: Sonucun eşzamanlı işlemlerin timing/sirasına bağlı olarak yanlış değişmesi problemidir.
- FinWallet'ta kullanım: Aynı wallet'tan paralel 600+600 harcama, aynı idempotency key veya refresh token race'i örneklerdir.
- Neden / çözdüğü problem: Financial correctness'in multi-instance ortamda da deterministik kalmasını gerektirir.
- Not / trade-off: Unit test tek-thread race'i kanıtlamaz; gerçek DB concurrency testleri gerekir.

RDB Snapshot

- Tanım: Redis memory state'inin belirli anda disk snapshot olarak kaydedilmesidir.
- FinWallet'ta kullanım: Docker runbook'ta BGSAVE ile snapshot istenebilir.
- Neden / çözdüğü problem: Local recovery/debug için ek persistence seçeneği sağlar.
- Not / trade-off: Point-in-time snapshot olduğundan son write'ları kaçırabilir ve finansal ledger yerine kullanılmaz.

Redis

- Tanım: Memory-first key/value veri store'udur; TTL, hızlı counter ve geçici coordination için uygundur.
- FinWallet'ta kullanım: FinWallet OTP TTL, temporary counters, fraud velocity ve hot/transient support state için kullanır.
- Neden / çözdüğü problem: Düşük latency gerektiren geçici state'i MSSQL yükünden ayırır.
- Not / trade-off: Wallet balance veya ledger için authoritative değildir; Redis kaybı finansal truth kaybı olmamalıdır.

Redis AOF

- Tanım: Redis write komutlarını append-only file'a yazarak restart sonrası state recover etmeye yarayan persistence modudur.
- FinWallet'ta kullanım: Docker Redis local ortamında transient support state'i daha dayanıklı tutmak için AOF etkinleştirilebilir.
- Neden / çözdüğü problem: Container restart sonrası bazı transient state'in kaybını azaltır.
- Not / trade-off: Redis yine financial source of truth olmaz; AOF MSSQL backup'ın alternatifi değildir.

Rollback

- Tanım: Transaction içindeki değişiklikleri commit edilmeden geri almaktır.
- FinWallet'ta kullanım: Ledger imbalance, insufficient balance veya SQL error olduğunda financial posting rollback olur.
- Neden / çözdüğü problem: Partial DB state bırakılmasını engeller.
- Not / trade-off: Commit edilmiş dış provider hareketini rollback edemez; onun için compensation gerekir.

Schema

- Tanım: Veritabanındaki tablo, kolon, constraint, index ve ilişkilerin yapısal tanımıdır.
- FinWallet'ta kullanım: FinWallet schema scriptleri finansal ve auth tablolarını versioned şekilde oluşturur.
- Neden / çözdüğü problem: Data correctness'i uygulama kodundan bağımsız DB kurallarıyla destekler.

- Not / trade-off: Schema değişikliği backward compatibility ve migration planı gerektirir.

SchemaVersions

- Tanım: Hangi DB migration'larının uygulanmış olduğunu kaydeden version tablosu/pattern'idir.
- FinWallet'ta kullanım: Init job script uygulanmadan önce version kaydına bakar.
- Neden / çözdüğü problem: Container restart'ında migration'ın yanlışlıkla tekrar çalışmasını önler.
- Not / trade-off: Migration dosyası değiştirildiyse eski version kaydı bunu algılamaz; uygulanmış migration immutable tutulmalıdır.

Serializable Isolation

- Tanım: SQL transaction'larda en güçlü standart isolation seviyelerinden biridir; concurrent sonucu serial execution'a yakın hale getirir.
- FinWallet'ta kullanım: Critical financial races için bazı atomic posting akışlarında kullanılabilir.
- Neden / çözdüğü problem: Overspend ve phantom-style race'leri güçlü biçimde sınırlar.
- Not / trade-off: Lock contention/deadlock maliyeti daha yüksektir; sadece gereken boundary'de kullanılmalıdır.

Source of Truth

- Tanım: Bir veri için doğru kabul edilen authoritative sistemdir.
- FinWallet'ta kullanım: Finansal state için MSSQL; dış banka hareketinin dış dünyadaki truth'u için bank provider kendi alanında source of truth'tür.
- Neden / çözdüğü problem: Çelişen cache/provider snapshot'larında hangi sistemin esas olduğunu belirler.
- Not / trade-off: Birden fazla source of truth tanımlamak reconciliation ve ownership'i belirsiz hale getirir.

Transient State

- Tanım: Kaybolması halinde finansal truth'u bozmayan kısa ömürlü/geçici veridir.
- FinWallet'ta kullanım: OTP TTL, fraud velocity counters veya cache-like bilgiler Redis'te tutulabilir.
- Neden / çözdüğü problem: Hızlı erişim sağlar ve durable DB'yi gereksiz yükten korur.
- Not / trade-off: Kaybolduğunda yeniden üretilebilmeli veya güvenli şekilde expire olabilmelidir.

TTL (Time To Live)

- Tanım: Bir cache/key verisinin ne kadar süre sonra otomatik expire olacağını belirleyen süredir.
- FinWallet'ta kullanım: Registration OTP ve bazı transient Redis state'leri TTL ile sınırlanır.
- Neden / çözdüğü problem: Geçici doğrulama verisinin sonsuza kadar kalmasını engeller.
- Not / trade-off: TTL finansal transaction retention'ı değildir; durable financial data expire edilmez.

Unique Constraint

- Tanım: Belirli kolon veya kolon kombinasyonunun duplicate olmasını DB seviyesinde engelleyen kuraldır.
- FinWallet'ta kullanım: Idempotency key, inbox Source+MessageId veya domain-specific uniqueness için kullanılır.

- Neden / çözdüğü problem: Race condition sırasında iki instance aynı kaydı oluşturmaya çalışsa bile tek winner sağlar.
- Not / trade-off: Sadece uygulama SELECT then INSERT kontrolüne güvenmekten daha güvenlidir.

UTC / DateTimeOffset

- Tanım: Zaman bilgisini timezone bağımsız veya offset bilgili biçimde saklama yaklaşımıdır.
- FinWallet'ta kullanım: CreatedAt, CompletedAt, ReviewedAt gibi timestamps UTC/DateTimeOffset kullanır.
- Neden / çözdüğü problem: Farklı servis/ülke saatlerinde sıralama ve audit tutarlılığını artırır.
- Not / trade-off: Business cutoff gibi yerel takvim kuralları yine country/timezone context'iyile hesaplanmalıdır.

5. Güvenilirlik, Mesajlaşma ve Dağıtık Sistem (22 terim)

At-Least-Once Delivery

- Tanım: Bir mesajın en az bir kez teslim edilmesini hedefleyen, duplicate teslimata izin veren messaging guarantee'dir.
- FinWallet'ta kullanım: Outbox worker hata durumunda mesajı retry eder; aynı message birden fazla delivery attempt görebilir.
- Neden / çözdüğü problem: Message loss yerine duplicate ihtimalini kabul eder.
- Not / trade-off: Exactly-once gibi davranış için idempotent consumer/deduplication gerekir.

Background Worker / Hosted Service

- Tanım: HTTP request'ten bağımsız sürekli veya periyodik arka plan işi çalıştıran process bileşenidir.
- FinWallet'ta kullanım: BankMoneyMovementBackgroundService ve OutboxDispatchBackgroundService bu rolü taşır.
- Neden / çözdüğü problem: Uzun/pending provider akışlarını client request'ine bağlamadan ilerletir.
- Not / trade-off: Worker state memory'de değil durable DB'de olmalıdır ki restart güvenli olsun.

Backoff

- Tanım: Retry'ler arasında bekleme süresini artırarak downstream servise sürekli yük bindirmemektir.
- FinWallet'ta kullanım: Outbox failure state sonraki denemeyi ileri zamana planlayabilir.
- Neden / çözdüğü problem: Dependency outage sırasında retry storm'u azaltır.
- Not / trade-off: Backoff tek başına retry limit/dead-letter kararının yerini almaz.

Callback

- Tanım: Dış provider'ın bir işlemin durumunu daha sonra FinWallet'a bildirdiği server-to-server çağrıdır.
- FinWallet'ta kullanım: Internal bank callback endpoint'i Pending/Completed/Failed provider state'ini Inbox üzerinden işler.
- Neden / çözdüğü problem: Uzun süren provider işlemlerini client polling'e bağımlı bırakmaz.
- Not / trade-off: Callback authentication, dedupe ve replay-safe finalization şarttır.

Claim / Lease

- Tanım: Bir worker'ın belirli işi geçici süreyle kendine ayırması ve diğer worker'ların aynı işi eşzamanlı almamasıdır.
- FinWallet'ta kullanım: Outbox dispatcher SQL atomik claim ile bir message batch'ini sahiplenir.
- Neden / çözdüğü problem: Multi-instance worker ortamında duplicate concurrent send riskini azaltır.
- Not / trade-off: Worker lease sonrası crash ederse row yeniden claim edilebilir olmalıdır.

Compensating Transaction

- Tanım: Önceki finansal transaction'ı silmek yerine ters etkili yeni transaction yaratılmasıdır.
- FinWallet'ta kullanım: Refund ve Reversal original journal'ı mutate etmeden karşıt journal/transaction oluşturur.
- Neden / çözdüğü problem: Audit trail'i korur ve geçmişin immutable kalmasını sağlar.
- Not / trade-off: Her transaction türü aynı correction yöntemini kullanmaz; external bank operation provider compensation gerektirebilir.

Compensation

- Tanım: Distributed akışta daha önce gerçekleşmiş bir etkinin karşıt business işlemiyle telafi edilmesidir.
- FinWallet'ta kullanım: Provider fail sonrası blocked balance release veya external movement sonrası local mismatch için correction/recovery tasarlanır.
- Neden / çözdüğü problem: DB rollback'in dış sistemdeki tamamlanmış işlemi geri alamadığı durumları yönetir.
- Not / trade-off: Compensation rollback değildir; yeni, audit edilebilir bir business action'dır.

Deduplication

- Tanım: Aynı logical event/request/message'in ikinci kez etkili işlenmesini engellemektir.
- FinWallet'ta kullanım: Inbox unique key, idempotency key ve terminal state kontrolleri farklı katmanlarda dedupe sağlar.
- Neden / çözdüğü problem: Network retry ve provider duplicate'larını güvenli hale getirir.
- Not / trade-off: Dedupe key doğru seçilmezse farklı gerçek işlemler yanlışlıkla aynı kabul edilebilir.

Eventual Consistency

- Tanım: Farklı sistemlerin aynı anda değil bir süre sonra tutarlı hale gelmesini kabul eden modeldir.
- FinWallet'ta kullanım: Para MSSQL'de commit olduktan sonra notification Outbox ile daha sonra gönderilebilir; provider callback/reconciliation da zaman farkıyla gelir.
- Neden / çözdüğü problem: Distributed transaction olmadan güvenilir entegrasyon sağlar.
- Not / trade-off: Hangi verinin anında consistent, hangisinin eventual olduğu açıkça tanımlanmalıdır.

Exactly-Once Effect

- Tanım: Distributed sistemde mesaj birden çok kez teslim edilse bile business etkinin tek kez oluşması hedefidir.
- FinWallet'ta kullanım: FinWallet bunu gerçek 'exactly-once transport' yerine idempotency + Inbox + terminal-state guards ile yaklaşıp.
- Neden / çözdüğü problem: Pratikte güvenli para hareketi sağlar.
- Not / trade-off: Network seviyesinde mutlak exactly-once iddiası yapılmaz; effect-level dedupe uygulanır.

ExternalTransactionId

- Tanım: Dış bank/provider tarafından üretilen transaction kimliğidir.
- FinWallet'ta kullanım: Bank movement status query, callback dedupe/finalization ve reconciliation'da local FinancialTransaction ile ilişkilendirilir.
- Neden / çözdüğü problem: FinWallet transaction ID ile provider transaction ID'yi birbirinden ayırır.
- Not / trade-off: Correlation ID değildir; provider'ın business operation identity'sidir.

Idempotency

- Tanım: Aynı mantıksal request birden fazla kez gönderildiğinde finansal etkinin yalnız bir kez uygulanması özelliğidir.
- FinWallet'ta kullanım: Transfer, Purchase, BankDeposit/Withdrawal ve correction komutlarında durable Idempotency-Key kullanılır.
- Neden / çözdüğü problem: Timeout/retry/network belirsizliğinde duplicate para hareketini engeller.
- Not / trade-off: Aynı key + aynı payload replay edilir; aynı key + farklı payload conflict olmalıdır.

Idempotency-Key

- Tanım: Client'in aynı finansal komutu tekrar gönderdiğini server'ın anlayabilmesi için verdiği stable request anahtarıdır.
- FinWallet'ta kullanım: Money-moving public POST endpoint'lerinde header olarak zorunludur.
- Neden / çözdüğü problem: Client retry ile yeni transaction yaratılması yerine önceki sonucu replay etmeyi sağlar.
- Not / trade-off: Correlation ID ile aynı değildir; request tekrar güvenliği için özel semantik taşır.

Inbox Pattern

- Tanım: Dışarıdan gelen mesaj/callback'ı önce durable kaydedip unique message identity ile duplicate processing'i engelleyen pattern'dir.
- FinWallet'ta kullanım: FakeBank callback'ları Source + MessageId ile Inbox'ta dedupe edilir.
- Neden / çözdüğü problem: Provider aynı callback'i yüzlerce kez gönderse bile finansal finalization'ın tekrar uygulanmasını engeller.
- Not / trade-off: Consumer business operation'ın kendisi de replay-safe olmalıdır; Inbox tek başına crash aralığını tamamen çözmez.

Non-Retryable Error

- Tanım: Aynı request'i tekrar denemenin anlamlı olmadığı terminal/business hatadır.
- FinWallet'ta kullanım: Invalid account, permanent provider reject veya insufficient funds gibi durumlarda operation Failed olur ve blocked amount release edilebilir.
- Neden / çözdüğü problem: Sonsuz retry ve stuck blocked-balance riskini önler.
- Not / trade-off: Yanlış classification availability kaybı veya duplicate effect riski yaratabilir.

Outbox Pattern

- Tanım: Business transaction ile gönderilecek mesajı aynı database transaction içinde kaydedip dış gönderimi sonradan worker ile yapan reliability pattern'idir.
- FinWallet'ta kullanım: Purchase/Bank movement completion gibi akışlar notification Outbox row'u yazar; worker FakeCommunication'a yollar.

- Neden / çözdüğü problem: Para commit olup process crash olduğunda notification'ın tamamen kaybolmasını önler.
- Not / trade-off: Genellikle at-least-once delivery sağlar; consumer/message operation idempotent olmalıdır.

Provider Idempotency Key

- Tanım: FinWallet'ın external provider'a aynı logical operation'ı tekrar gönderdiğinde duplicate side effect'i önlemek için kullandığı stable key'dir.
- FinWallet'ta kullanım: Bank account opening ve bank money movement request'leri provider requestKey taşır.
- Neden / çözdüğü problem: Client idempotency'den bağımsız olarak downstream duplicate protection sağlar.
- Not / trade-off: Provider gerçekten key semantiğini garanti etmiyorsa FinWallet blind retry yapmaz.

Replay

- Tanım: Daha önce tamamlanmış idempotent işlemin aynı sonucunun yeniden döndürülmesidir.
- FinWallet'ta kullanım: Transfer/Purchase handler önce completed replay store'a bakar; fraud/provider tekrar çağrılmadan sonuç dönebilir.
- Neden / çözdüğü problem: Duplicate request'in para ve dış servis maliyetini yeniden üretmesini engeller.
- Not / trade-off: Replay response orijinal completion timestamp ve transaction identity'yi korumalıdır.

Request Hash

- Tanım: Idempotency key ile birlikte payload'ın aynı olup olmadığını kanıtlamak için canonical request'ten üretilen hash'tir.
- FinWallet'ta kullanım: Fraud event/idempotency akışında source/destination/amount gibi alanlar canonicalize edilip SHA-256 hashlenebilir.
- Neden / çözdüğü problem: Aynı key'in farklı payload ile kötüye kullanılmasını yakalar.
- Not / trade-off: Canonicalization deterministic olmalıdır; field order/format farkı yanlış conflict üretmemelidir.

Retry

- Tanım: Geçici hata sonrası aynı operasyonu tekrar denemektir.
- FinWallet'ta kullanım: Outbox communication ve retryable bank status processing kontrollü retry edilebilir.
- Neden / çözdüğü problem: Transient network/provider sorunlarında availability'yi artırır.
- Not / trade-off: Financial POST körlemesine retry edilmez; provider idempotency garanti edilmeden duplicate para hareketi riski vardır.

Retryable Error

- Tanım: Aynı operasyon daha sonra tekrar denendiğinde başarılı olma ihtimali olan geçici hatadır.
- FinWallet'ta kullanım: Provider timeout/network interruption veya temporary communication outage buna örnek olabilir.
- Neden / çözdüğü problem: Worker'ın pending/backoff state'inde kalmasını sağlar.
- Not / trade-off: Business reject veya insufficient funds retryable değildir; error classification doğru yapılmalıdır.

Terminal State

- Tanım: Bir transaction'ın artık normal lifecycle içinde başka business processing gerektirmeyen son durumudur.
- FinWallet'ta kullanım: Completed ve bazı Failed/Denied state'ler terminal kabul edilir.
- Neden / çözdüğü problem: Duplicate callback/retry geldiğinde yeniden para hareketi yapılmasını engelleyen guard sağlar.
- Not / trade-off: Manual correction terminal transaction üzerinde yeni child transaction yaratabilir; original state yine mutata edilmez.

6. Finans, Muhasebe ve Bankacılık (57 terim)

Asset

- Tanım: Şirketin sahip olduğu veya kontrol ettiği ekonomik değerleri temsil eden muhasebe hesabı türüdür.
- FinWallet'ta kullanım: BANK-SETTLEMENT:TRY FinWallet ledger'ında bankacılık tarafındaki karşılık varlığını temsil eder.
- Neden / çözdüğü problem: Wallet liability'nin karşılığının nerede tutulduğunu muhasebe denkleminde görünür kılar.
- Not / trade-off: Mevcut FakeBank modelinde ayrı gerçek FinWallet omnibus hesabı tam modellenmemiştir; settlement asset şimdilik FinWallet muhasebe abstraction'ıdır.

Atomic Posting

- Tanım: Posting'in tüm finansal bileşenlerinin tek DB transaction içinde ya hep ya hiç uygulanmasıdır.
- FinWallet'ta kullanım: Balance + FinancialTransaction + Journal/Entries + idempotency + gerektiğinde Outbox aynı commit'te tutulur.
- Neden / çözdüğü problem: Partial money movement ve ledger/balance ayrışmasını önler.
- Not / trade-off: External provider atomic posting kapsamına dahil değildir.

Available Balance

- Tanım: Wallet'ta müşterinin hemen harcayabileceği/transfer edebileceği bakiyedir.
- FinWallet'ta kullanım: Transfer ve Purchase source wallet'tan available balance düşer.
- Neden / çözdüğü problem: Overspend kontrolünün temelidir.
- Not / trade-off: Pending external withdrawal gibi akışlarda para available'dan blocked'a taşınabilir.

Bank Account

- Tanım: Dış banka/provider tarafında müşteriye ait gerçek/harici hesap modelidir.
- FinWallet'ta kullanım: FinWallet BankAccount kaydı FakeBank externalAccountId/IBAN/currency/status ile wallet'a bağlanır.
- Neden / çözdüğü problem: Bankadaki para ile FinWallet wallet bakiyesini ayrı modeller ve funding/withdrawal akışına boundary sağlar.
- Not / trade-off: BankAccount balance ile Wallet balance eşit olmak zorunda değildir.

Bank Reference

- Tanım: Bank/provider işlemine ait external referans bilgisidir.

- FinWallet'ta kullanım: Transaction history/troubleshooting sırasında ExternalTransactionId veya provider-specific reference ile birlikte kullanılabilir.
- Neden / çözdüğü problem: Customer support/reconciliation'da iki sistemde aynı hareketi bulmayı kolaylaştırır.
- Not / trade-off: Internal transaction ID ile karıştırılmamalıdır.

Bank Settlement Asset

- Tanım: FinWallet ledger'ında external bank tarafındaki müşteri fonu karşılığını temsil eden asset account abstraction'ıdır.
- FinWallet'ta kullanım: Bank->Wallet deposit'te debit edilir; Wallet->Bank withdrawal final olduğunda credit edilerek azalır.
- Neden / çözdüğü problem: Toplam wallet liability'nin bankacılık tarafındaki karşılığını muhasebe olarak izlemeyi sağlar.
- Not / trade-off: FakeBank'te ayrı FinWallet omnibus/safeguarding account henüz tam modellenmediğinden bu account gerçek provider hesabının birebir kaydı değildir.

Bank Statement

- Tanım: Dış banka hesabındaki tamamlanmış hareketlerin kronolojik listesi veya ekstresidir.
- FinWallet'ta kullanım: IBankProvider.GetStatementAsync reconciliation için provider statement satırlarını getirir.
- Neden / çözdüğü problem: FinWallet local bank movement kayıtları ile dış provider truth'unu karşılaştırmayı sağlar.
- Not / trade-off: Statement query financial mutation yapmaz; reconciliation read source'tur.

Bank-Settlement Reconciliation

- Tanım: Internal bank transaction kayıtlarının settlement ledger etkisiyle uyumunu kontrol etmektir.
- FinWallet'ta kullanım: Completed BankDeposit/Withdrawal transaction amount/status ile bank-settlement entries karşılaştırılır.
- Neden / çözdüğü problem: Local transaction state ile accounting state ayrışmasını yakalar.
- Not / trade-off: External provider statement reconciliation'dan farklı scope'tur.

BankDeposit

- Tanım: FinWallet açısından dış banka hesabından müşterinin digital wallet'ına para yükleme işlemidir.
- FinWallet'ta kullanım: Client POST /bank-movements/deposits çağırır; provider adapter FakeBank tarafında customer bank account'tan para çekmek için provider Withdrawal yönüne map eder; local ledger settlement asset + wallet liability artırır.
- Neden / çözdüğü problem: Bankadaki parayı FinWallet içi kullanılabilir bakiyeye dönüştürür.
- Not / trade-off: İsim provider yönüyle karıştırılmamalıdır: FinWallet Deposit = wallet'a giriş; FakeBank'te aynı ekonomik hareket customer bank account'tan withdrawal'dır.

BankWithdrawal

- Tanım: FinWallet açısından digital wallet'tan dış banka hesabına para gönderme işlemidir.
- FinWallet'ta kullanım: Amount önce available'dan blocked'a alınabilir; cutoff/provider işlenir; tamamlanınca wallet liability ve settlement asset azalır, provider tarafında customer bank account credit edilir.
- Neden / çözdüğü problem: Pending bank işlemi sırasında aynı paranın ikinci kez harcanmasını engeller.

- Not / trade-off: Provider failure'da blocked release/compensation doğru yapılmalıdır.

Blocked Balance

- Tanım: Geçici olarak kullanıma kapatılmış fakat henüz final settlement ile wallet'tan tamamen çıkmamış bakiyedir.
- FinWallet'ta kullanım: Wallet->Bank withdrawal provider pending iken amount available'dan blocked'a taşınır.
- Neden / çözdüğü problem: Aynı paranın başka transfer/purchase ile ikinci kez harcanmasını engeller.
- Not / trade-off: Provider terminal failure verirse blocked amount release edilmelidir; aksi halde para stuck olur.

Business Day / Business Calendar

- Tanım: Bankacılık işlemlerinin çalıştığı gün/tatil kurallarını tanımlayan takvimdir.
- FinWallet'ta kullanım: FakeCutoff weekend/holiday/processing date mantığını simüle eder.
- Neden / çözdüğü problem: Calendar-day ile bank-processing-day farkını doğru modellemeyi sağlar.
- Not / trade-off: Production'da resmi holiday calendar veri kaynağı ve timezone yönetimi gerekir.

Campaign

- Tanım: Belirli merchant/customer/tutar koşullarında indirim uygulayan business kuralıdır.
- FinWallet'ta kullanım: FakeCampaign eligibility, discount amount ve sponsor type döndürür; accounting FinWallet'ta yapılır.
- Neden / çözdüğü problem: Promotion logic'ini finansal posting'den ayırır.
- Not / trade-off: Provider sadece hesaplama/karar verir; ledger'a yazma yetkisi yoktur.

Campaign Expense

- Tanım: Platformun finanse ettiği kampanya indirimini şirket maliyeti olarak temsil eden expense account'ıdır.
- FinWallet'ta kullanım: Örneğin 200 TRY purchase, 20 TRY platform discount: customer liability debit 180, campaign expense debit 20, merchant payable credit 200.
- Neden / çözdüğü problem: İndirimin ekonomik kaynağını ledger'da açık tutar.
- Not / trade-off: Merchant sponsorlu discount'ta aynı expense hesabı kullanılmayabilir; payable net tutara göre oluşabilir.

Credit

- Tanım: Double-entry muhasebede journal'ın sağ tarafındaki entry yönüdür.
- FinWallet'ta kullanım: Liability hesabını artırabilir, asset hesabını azaltabilir; BankDeposit'te customer wallet liability credit edilir.
- Neden / çözdüğü problem: Debit ile birlikte journal balance sağlar.
- Not / trade-off: Credit her zaman 'para geldi' anlamına gelmez; account türüne göre yorumlanır.

Currency

- Tanım: Paranın hangi para biriminde olduğunu belirleyen kod/enum'dur.
- FinWallet'ta kullanım: TRY, USD, EUR wallet ve account'larda currency boundary oluşturur.
- Neden / çözdüğü problem: Cross-currency yanlış posting'i engeller.

- Not / trade-off: V1'de FX yoktur; farklı currency wallet'lar arasında otomatik dönüşüm yapılmaz.

Customer Wallet Liability

- Tanım: Müşterinin wallet bakiyesini FinWallet'ın müşteriye borcu olarak temsil eden ledger liability account'ıdır.
- FinWallet'ta kullanım: BankDeposit'te credit ile artar; transfer source veya purchase'ta debit ile azalır.
- Neden / çözdüğü problem: Wallet balance'ın ekonomik anlamını muhasebe olarak doğru gösterir.
- Not / trade-off: Wallet table current balance ile ledger liability toplamının reconciliation ile uyumlu olması beklenir.

Cutoff

- Tanım: Banka işleminin aynı iş gününde mi yoksa sonraki business day'de mi işleneceğini etkileyen saat/kural sınırıdır.
- FinWallet'ta kullanım: FakeCutoff business calendar ve transaction type'a göre karar verir.
- Neden / çözdüğü problem: After-hours bank withdrawal gibi işlemleri Scheduled state'e almayı sağlar.
- Not / trade-off: Cutoff sabit tek saat değildir; ülke/currency/işlem tipine göre değişebilir.

Debit

- Tanım: Double-entry muhasebede journal'ın sol tarafındaki entry yönüdür.
- FinWallet'ta kullanım: Asset hesabını artırabilir, liability hesabını azaltabilir; örneğin BankDeposit'te settlement asset debit edilir, transfer source wallet liability debit edilerek azaltılır.
- Neden / çözdüğü problem: Account türüne göre ekonomik yönü açık şekilde modellemeyi sağlar.
- Not / trade-off: Debit her zaman 'para çıktı' demek değildir.

Digital Wallet

- Tanım: FinWallet içinde müşterinin kullanabilir dijital para bakiyesini temsil eden iç finansal hesaptır.
- FinWallet'ta kullanım: Her wallet currency bazında AvailableBalance/BlockedBalance ve ledger liability ilişkisi taşır.
- Neden / çözdüğü problem: Transfer, Purchase, Refund gibi FinWallet içi para hareketlerinin hızlı ve bağımsız yürütülmesini sağlar.
- Not / trade-off: BankAccount ile aynı hesap değildir; wallet bakiyesi FinWallet'ın müşteriye karşı yükümlülüğünü temsil eder.

Discount

- Tanım: Purchase original amount'tan düşülen kampanya tutarıdır.
- FinWallet'ta kullanım: OriginalAmount ve DiscountAmount transaction detail/history'de tutulabilir.
- Neden / çözdüğü problem: Customer'ın ödediği net tutar ile merchant'ın hak ettiği tutarı ayırır.
- Not / trade-off: İndirimi kimin finanse ettiği sponsor modeline göre ledger entry değişir.

Double-Entry Bookkeeping

- Tanım: Her finansal olayın en az iki muhasebe hesabını etkilediği ve toplam debit = toplam credit olduğu muhasebe yöntemidir.
- FinWallet'ta kullanım: FinWallet transfer, bank deposit, purchase, refund/reversal posting'lerinde kullanır.

- Neden / çözdüğü problem: Para 'yoktan oluştu mu/kayboldu mu?' hatalarını balance equation ile yakalamayı sağlar.
- Not / trade-off: Debit/credit işareti 'iyi/kötü' veya basit artı/eksi değildir; account türüne göre anlam değişir.

Expense

- Tanım: Şirketin belirli faaliyet nedeniyle katlandığı maliyeti temsil eden muhasebe hesap türüdür.
- FinWallet'ta kullanım: Platform sponsorlu campaign discount'ta CAMPAIGN-EXPENSE benzeri hesap debit edilebilir.
- Neden / çözdüğü problem: İndirimin merchant veya customer bakiyesinden gizlice düşmek yerine ekonomik sponsorunu açıklar.
- Not / trade-off: Campaign sponsor tipi accounting posting'i değiştirir.

External Bank Statement Reconciliation

- Tanım: FinWallet local bank movement kayıtlarını FakeBank/gerçek bank statement ile karşılaştırmaktır.
- FinWallet'ta kullanım: ExternalTransactionId ve amount/status/reference üzerinden mismatch tespit edilir.
- Neden / çözdüğü problem: Provider'da tamamlanan ama local'de eksik veya tersi durumları bulur.
- Not / trade-off: Provider call SQL transaction dışında yapılır; result sonra durable issue/run'a yazılır.

FinancialTransaction

- Tanım: Bir business para hareketinin lifecycle, amount, type, status ve referanslarını taşıyan durable işlem kayıdır.
- FinWallet'ta kullanım: Transfer, BankDeposit, BankWithdrawal, Purchase, Refund, Reversal gibi tipler tek transaction modelinde izlenir.
- Neden / çözdüğü problem: API/business işlem kimliği ile muhasebe journal'ını birbirinden ayırarak lifecycle takibi sağlar.
- Not / trade-off: FinancialTransaction ledger'ın kendisi değildir; transaction 'ne oldu?', ledger 'muhasebe etkisi neydi?' sorusunu cevaplar.

FinancialTransactionStatus

- Tanım: FinancialTransaction'ın lifecycle durumunu temsil eden state'tir.
- FinWallet'ta kullanım: Scheduled, Pending, Completed, Failed gibi durumlar bank/financial flows'da kullanılır.
- Neden / çözdüğü problem: Worker, callback ve history API'nin aynı transaction'ın hangi aşamada olduğunu anlamasını sağlar.
- Not / trade-off: Terminal state'ler tekrar posting'i engellemek için önemlidir.

FinancialTransactionType

- Tanım: FinancialTransaction'ın hangi business hareketini temsil ettiğini belirleyen enum/type'tir.
- FinWallet'ta kullanım: WalletTransfer, BankDeposit, BankWithdrawal, Purchase, Refund, Reversal gibi değerler bulunur.
- Neden / çözdüğü problem: Posting/correction/history logic'inin işlem semantiğini anlamasını sağlar.
- Not / trade-off: DB numeric contract ile code enum değerleri uyumlu kalmalıdır.

IBAN

- Tanım: Banka hesabını uluslararası standart formatta tanımlayan hesap numarasıdır.
- FinWallet'ta kullanım: FakeBank account açıldığında örnek IBAN üretir ve FinWallet BankAccount'a kaydedilir.
- Neden / çözdüğü problem: Dış banka hesabını user-facing/reference olarak tanımlar.
- Not / trade-off: Simulator IBAN'ı gerçek bankacılık clearing sistemine bağlı değildir.

Ledger

- Tanım: FinWallet içindeki tüm finansal hareketleri hesaplar bazında debit/credit kayıtlarıyla immutable şekilde izleyen muhasebe defteridir.
- FinWallet'ta kullanım: Her completed financial posting bir LedgerJournal ve en az iki LedgerEntry üretir; toplam Debit toplam Credit'e eşit olmalıdır.
- Neden / çözdüğü problem: Sadece 'şu an bakiye kaç?' değil, 'bu bakiye nasıl oluştu?' sorusunun audit edilebilir cevabını verir.
- Not / trade-off: Wallet tablosu current projection'dır; Ledger geçmişin muhasebe kaynağıdır. Original entry silinmez, correction ters kayıtla yapılır.

Ledger Account

- Tanım: Ledger içindeki debit/credit hareketlerinin toplandığı muhasebe hesabıdır.
- FinWallet'ta kullanım: BANK-SETTLEMENT:TRY, WALLET-LIABILITY:<walletId>, MERCHANT-PAYABLE:<merchant> gibi logical account'lar kullanılır.
- Neden / çözdüğü problem: Her para hareketinin hangi ekonomik hesaba ait olduğunu açıklar.
- Not / trade-off: Ledger Account, müşterinin BankAccount entity'siyle aynı kavram değildir.

Ledger Entry

- Tanım: Bir LedgerJournal içindeki tek debit veya credit satırıdır.
- FinWallet'ta kullanım: Account, side (Debit/Credit), amount/currency ve reference bilgisi taşır.
- Neden / çözdüğü problem: Journal'ın ekonomik etkisini atomik hesap satırlarına ayırır.
- Not / trade-off: Entry tek başına transaction'ın tamamını anlatmaz; journal ile birlikte okunur.

Ledger Journal

- Tanım: Tek bir finansal olayın tüm debit/credit entry'lerini birlikte gruplayan kayıt başlığıdır.
- FinWallet'ta kullanım: Transfer, deposit, purchase veya correction için bir journal oluşturulur; journal altındaki entries dengeli olmalıdır.
- Neden / çözdüğü problem: Bir işlemin muhasebe etkilerini tek business event olarak audit etmeyi sağlar.
- Not / trade-off: Journal mutate edilmez; reversal/refund yeni journal üretir.

Liability

- Tanım: Şirketin başka bir tarafa karşı borç/yükümlülüğünü temsil eden muhasebe hesabı türüdür.
- FinWallet'ta kullanım: Müşterinin wallet bakiyesi WALLET-LIABILITY:<wallet> ile FinWallet'ın müşteriye karşı yükümlülüğü olarak modellenir.
- Neden / çözdüğü problem: Wallet bakiyesini şirketin 'kendi parası' gibi yanlış modellemeyi engeller.
- Not / trade-off: Liability arttığında normalde credit, azaldığında debit edilir.

Merchant

- Tanım: Müşterinin ürün/hizmet satın aldığı ticari tarafı temsil eder.
- FinWallet'ta kullanım: Purchase request merchantId taşır; Campaign ve Fraud değerlendirmeleri merchant context kullanabilir.
- Neden / çözdüğü problem: Wallet harcamasının karşı tarafını ve payable hesabını belirler.
- Not / trade-off: V1 merchant onboarding/settlement sistemi tam kapsamlı ödeme kuruluşu modeli değildir.

Merchant Payable

- Tanım: Purchase sonrası FinWallet'ın merchant'a borçlandığı tutarı temsil eden liability account'dır.
- FinWallet'ta kullanım: Purchase journal'ında net/gross campaign sponsor modeline göre merchant payable credit edilir.
- Neden / çözdüğü problem: Customer wallet debit'ini merchant settlement obligation'a dönüştürür.
- Not / trade-off: Merchant payable'ın gerçek bank settlement'ı v1'de ayrı operasyonel kapsam olabilir.

Money

- Tanım: Amount ve Currency'yi tek domain değeri olarak birlikte taşıyan finansal Value Object'tur.
- FinWallet'ta kullanım: Transfer, purchase, bank movement ve fraud evaluation amount'ları Money semantiğiyle işlenir.
- Neden / çözdüğü problem: 100 TRY ile 100 USD'nin yanlışlıkla aynı değer gibi kullanılmasını engeller.
- Not / trade-off: Rounding/scale kuralları Money invariant'ının parçası olmalıdır.

Negative Balance

- Tanım: Wallet available/blocked veya ilgili finansal account'un iş kuralına aykırı biçimde sıfırın altına düşmesidir.
- FinWallet'ta kullanım: Customer wallet'ta negatif/overspend v1 invariant olarak yasaktır.
- Neden / çözdüğü problem: Kredi/overdraft özelliği olmadığı için finansal doğruluğu sade tutar.
- Not / trade-off: İleride credit product eklenirse ayrı limit/loan modeline ihtiyaç olur; mevcut invariant sessizce değiştirilmemelidir.

Omnibus Account

- Tanım: Bir fintech/payment kuruluşunun birçok müşterinin fonunu tek bank account'ta toplu tuttuğu hesap modelidir.
- FinWallet'ta kullanım: FinWallet'ta ayrı gerçek FakeBank omnibus account henüz tam modellenmemiştir; BANK-SETTLEMENT ledger asset bu ekonomik fikri muhasebe olarak temsil eder.
- Neden / çözdüğü problem: Müşteri başına fiziksel banka hesabı yerine iç ledger'ın müşteri dağılımını tutmasını açıklar.
- Not / trade-off: Production regülasyonu safeguarding/segregation ve legal ownership kuralları gerektirebilir.

Original Transaction

- Tanım: Refund/Reversal gibi correction'ın referans verdiği ilk completed financial transaction'dır.
- FinWallet'ta kullanım: ParentTransactionId/correction checks original type/status/ownership üzerinden yapılır.
- Neden / çözdüğü problem: Correction'ın hangi ekonomik olayı terslediğini audit etmeyi sağlar.

- Not / trade-off: Original row status/history silinmez veya overwrite edilmez.

Overspend

- Tanım: Wallet'ta mevcut kullanılabilir bakiyeden daha fazla harcama/transfer yapılmasıdır.
- FinWallet'ta kullanım: Atomic SQL balance check/lock ile engellenir; concurrent request'lerde de negatif bakiye oluşmamalıdır.
- Neden / çözdüğü problem: Müşterinin olmayan parayı harcamasını ve ledger/balance insolvency'yi önler.
- Not / trade-off: Sadece application if(balance >= amount) kontrolü race-safe değildir.

Parent Transaction

- Tanım: Bir correction/child transaction'ın hangi önceki transaction'dan türediğini gösteren referanstır.
- FinWallet'ta kullanım: Refund/Reversal history response'unda ParentTransactionId bulunabilir.
- Neden / çözdüğü problem: Original ve compensating transaction zincirini görünür kılar.
- Not / trade-off: Parent relation accounting entry yerine geçmez; ledger journal'ları ayrıca dengeli olmalıdır.

Payable

- Tanım: Şirketin bir merchant/üçüncü tarafa ödemesi gereken tutarı temsil eden liability türüdür.
- FinWallet'ta kullanım: Purchase posting'inde MERCHANT-PAYABLE:<merchant> credit ile artar.
- Neden / çözdüğü problem: Customer harcamasını merchant'a olan settlement yükümlülüğüne dönüştürür.
- Not / trade-off: Merchant'a gerçek bank transferi ayrı settlement process olabilir.

Posting

- Tanım: Bir finansal business event'in wallet state ve double-entry ledger'a kalıcı olarak uygulanmasıdır.
- FinWallet'ta kullanım: Transfer posting source/destination balance, transaction, journal, entries ve idempotency sonucunu yazar.
- Neden / çözdüğü problem: Business kararını gerçek muhasebe state'ine dönüştürür.
- Not / trade-off: Fraud/provider external I/O posting transaction'ı açıkken yapılmaz.

Processing Date

- Tanım: Bank işleminin operasyonda işleneceği business date'tir.
- FinWallet'ta kullanım: FakeCutoff country/currency/transaction/time bilgisine göre hesaplayabilir.
- Neden / çözdüğü problem: Request tarihi ile gerçek bank iş günü işlenişini ayırır.
- Not / trade-off: UTC timestamp ile aynı şey değildir; business calendar semantiği taşır.

Provider Deposit / Withdrawal Direction

- Tanım: FakeBank hesabı açısından account bakiyesini artırma/azaltma yönleridir.
- FinWallet'ta kullanım: FakeBank Deposit bank account'u artırır; Withdrawal bank account'u azaltır. FinWallet BankDeposit bunun ters provider yönünü kullanabilir.
- Neden / çözdüğü problem: Aynı 'deposit' kelimesinin hangi sistem perspektifinden söylendiğini netleştirir.

- Not / trade-off: Adapter/ACL bu semantic farkı izole eder; domain'e provider enum'u sızdırılmaz.

Purchase

- Tanım: Müşterinin wallet bakiyesini kullanarak merchant'a ödeme oluşturduğu finansal işlemdir.
- FinWallet'ta kullanım: Fraud + Campaign değerlendirmesinden sonra wallet liability azalır, merchant payable artar, campaign sponsoruna göre ek expense/netting entry oluşur.
- Neden / çözdüğü problem: Wallet harcamasını merchant'a olan ekonomik yükümlülüğe dönüştürür.
- Not / trade-off: Bank account her purchase'ta anında hareket etmek zorunda değildir; internal ledger settlement modelidir.

Reconciliation

- Tanım: İki veya daha fazla finansal kayıt kaynağının beklenen/gerçek değerlerini karşılaştırıp farkları raporlama sürecidir.
- FinWallet'ta kullanım: Wallet<->Ledger, BankTransaction<->SettlementLedger ve FinWallet<->FakeBankStatement scope'ları vardır.
- Neden / çözdüğü problem: Silent drift, bug, missing callback veya provider mismatch'i tespit eder.
- Not / trade-off: FinWallet reconciliation otomatik bakiye düzeltmez; issue üretir ve finansal history'yi mutata etmez.

Reconciliation Issue

- Tanım: Reconciliation sırasında bulunan mismatch'i temsil eden durable kayıtlı problemdir.
- FinWallet'ta kullanım: ExpectedAmount/ActualAmount, transaction/wallet/bank reference ve PII içermeyen details saklanabilir.
- Neden / çözdüğü problem: Farkı görünür ve takip edilebilir hale getirir.
- Not / trade-off: Issue çözümü manual investigation/correction gerektirebilir; sistem sessizce overwrite etmez.

Reconciliation Run

- Tanım: Belirli scope için yapılan tek reconciliation çalışma örneğidir.
- FinWallet'ta kullanım: RunId, scope, status, started/completed time ve issue count durable kaydedilir.
- Neden / çözdüğü problem: Operasyon ekibinin hangi kontrolün ne zaman yapıldığını audit etmesini sağlar.
- Not / trade-off: Run sonucu otomatik correction anlamına gelmez.

Refund

- Tanım: Completed Purchase'ın tamamını veya tanımlı kısmını müşteriye geri veren correction işlemidir.
- FinWallet'ta kullanım: FinWallet v1 full Purchase refund için original journal'ı silmeden opposite journal ve yeni FinancialTransaction oluşturur.
- Neden / çözdüğü problem: Audit trail'i koruyarak müşteri wallet liability'sini geri artırır ve merchant/campaign etkisini tersler.
- Not / trade-off: Refund original purchase'ı mutata etmez; parent transaction ilişkisiyle izlenir.

Reversal

- Tanım: Hatalı/geri alınması gereken completed transaction'ın ekonomik etkisini tersleyen yeni transaction'dır.
- FinWallet'ta kullanım: Public reversal güvenli internal WalletTransfer için destination->source ters balance ve opposite ledger journal yaratır.
- Neden / çözdüğü problem: Immutable history korunurken yanlış para hareketi düzeltilebilir.
- Not / trade-off: External BankDeposit/Withdrawal doğrudan bu endpoint ile reverse edilmez; provider compensation gerekir.

Safeguarding

- Tanım: Müşteri fonlarının şirketin kendi operasyonel fonlarından hukuken/operasyonel olarak ayrıştırılarak korunması yaklaşımıdır.
- FinWallet'ta kullanım: FinWallet v1 teknik simülasyonunda tam regülasyon modeli yoktur; settlement asset/liability ayrımı kavramsal temel sağlar.
- Neden / çözdüğü problem: Wallet bakiyesinin şirket geliri olmadığını ve karşılığının korunması gerektiğini açıklar.
- Not / trade-off: Gerçek ürün için ülke regülasyonu, lisans, bank partner ve legal accounting tasarımı gerekir.

Settlement

- Tanım: Bir finansal işlemin taraflar arasındaki ekonomik yükümlülüğünün nihai olarak karşılanması sürecidir.
- FinWallet'ta kullanım: Bank movement processing/settlement date ve BANK-SETTLEMENT ledger account'ları bu kavramı kullanır.
- Neden / çözdüğü problem: Wallet içi book entry ile gerçek dış banka hareketini kavramsal olarak ayırır.
- Not / trade-off: Internal posting ile external bank settlement aynı anda olmak zorunda değildir; reconciliation gerekir.

Settlement Date

- Tanım: Bir bank/financial işlemin ekonomik olarak nihai settlement'a ulaşacağı business date'tir.
- FinWallet'ta kullanım: Cutoff provider processing/settlement tarihleri döndürür ve transaction detail'de saklanabilir.
- Neden / çözdüğü problem: Pending/scheduled işlemin ne zaman tamamlanmasının beklendiğini açıklar.
- Not / trade-off: Gerçek provider callback zamanı settlement date'ten farklı olabilir; reconciliation yine gerekir.

Sponsor

- Tanım: Kampanya indiriminin ekonomik maliyetini üstlenen tarafı belirtir.
- FinWallet'ta kullanım: Platform-sponsored ve merchant-sponsored senaryolarda posting farklıdır.
- Neden / çözdüğü problem: Discount'ın muhasebede hangi account'a yansıtacağını belirler.
- Not / trade-off: Sponsor bilgisi sadece UI etiketi değil accounting input'udur.

Wallet Transfer

- Tanım: Bir FinWallet wallet'ından başka wallet'a aynı currency'de internal para hareketidir.
- FinWallet'ta kullanım: Fraud allow sonrası source liability debit, destination liability credit ve balance update tek atomic posting'de yapılır.

- Neden / çözdüğü problem: Banka çağrısı olmadan internal book transfer sağlar.
- Not / trade-off: Source ve destination currency uyumlu olmalı; overspend ve idempotency korunmalıdır.

Wallet-Ledger Reconciliation

- Tanım: Wallet tablosundaki current balance ile ledger'dan türetilebilen ekonomik balance'ın karşılaştırılmasıdır.
- FinWallet'ta kullanım: Reconciliation scope'larından biridir.
- Neden / çözdüğü problem: Projection drift veya yanlış posting'i tespit eder.
- Not / trade-off: Mismatch bulunduğunda wallet doğrudan ledger'a eşitlenmez; issue oluşturulur.

7. Fraud ve Risk (17 terim)

Allow

- Tanım: Fraud değerlendirmesinde işlemin devam etmesine izin veren karardır.
- FinWallet'ta kullanım: Durable fraud state Allow/Approved ise transfer/purchase atomic posting'e geçer.
- Neden / çözdüğü problem: Risk kontrolünü geçtikten sonra business flow'u sürdürür.
- Not / trade-off: Allow 'işlem kesinlikle fraud değildir' anlamına gelmez; mevcut sinyallerle kabul edilebilir risk demektir.

Deny

- Tanım: Fraud değerlendirmesinde işlemin gerçekleştirilmemesi kararıdır.
- FinWallet'ta kullanım: Internal fraud doğrudan Deny verebilir veya combined policy final Deny üretebilir.
- Neden / çözdüğü problem: Riskli işlemin wallet/ledger posting'e geçmesini engeller.
- Not / trade-off: Deny reason code audit/operations için güvenli biçimde kaydedilmelidir; hassas model detail'i client'a sızdırılmamalıdır.

External Fraud Provider

- Tanım: Fraud kararının bir kısmını dış servisten alan entegrasyon boundary'sidir.
- FinWallet'ta kullanım: FakeFraud v1 simulator olarak Allow/Review/Deny, score ve reason code döndürür.
- Neden / çözdüğü problem: FinWallet'ın kendi fraud kurallarını dış risk motorundan bağımsız tutar.
- Not / trade-off: Provider unavailable olduğunda protected flow fail-closed davranır.

Fraud

- Tanım: Yetkisiz, kötü niyetli veya riskli finansal davranışı tespit/önleme problem alanıdır.
- FinWallet'ta kullanım: WalletTransfer ve Purchase öncesinde internal + external fraud değerlendirmesi çalışır.
- Neden / çözdüğü problem: Para hareketi gerçekleşmeden önce risk kararını business flow'a dahil eder.
- Not / trade-off: BankDeposit akışında mevcut v1 tasarımı fraud çağrısı yoktur; dokümanlarda sahte fraud adımı eklenmez.

Fraud Decision

- Tanım: Risk değerlendirmesi sonucunda verilen Allow, Review veya Deny karardır.
- FinWallet'ta kullanım: Internal ve external decision policy ile final karar oluşturulur.
- Neden / çözdüğü problem: Posting'in devam edip etmeyeceğini açık state olarak belirler.
- Not / trade-off: Decision ile HTTP status aynı şey değildir; Review 202 gibi API davranışına map edilir.

Fraud Decision Policy

- Tanım: Internal ve external fraud kararlarını tek final karara dönüştüren business policy'dir.
- FinWallet'ta kullanım: Örneğin herhangi bir Deny final Deny; Review kombinasyonları Review; yalnız güvenli kombinasyonlar Allow olabilir.
- Neden / çözdüğü problem: Karar birleştirmeyi handler içine dağılmış if'lerden çıkarır.
- Not / trade-off: Policy değişikliği risk appetite değişikliğidir ve test/audit gerektirir.

Fraud Score

- Tanım: Risk seviyesini numeric olarak ifade eden provider/model çıktısıdır.
- FinWallet'ta kullanım: FakeFraud score döndürebilir; FinWallet final kararını yalnız score'a körlemesine bağlamaz.
- Neden / çözdüğü problem: Threshold/policy kararlarına ek sinyal sağlar.
- Not / trade-off: Score calibration model/provider'a özgüdür ve zaman içinde değişebilir.

FraudEvent

- Tanım: Bir fraud değerlendirmesinin input identity, internal/external/final decision, reason code ve review state'ini durable saklayan kayıttır.
- FinWallet'ta kullanım: WalletTransfer/Purchase idempotent request'leri aynı fraud sonucunu replay etmek için FraudEvent kullanır.
- Neden / çözdüğü problem: Provider tekrar çağrısını ve decision kaybını önler; manual review audit trail sağlar.
- Not / trade-off: Raw sensitive payload/hash/PII internal response'larda gereksiz expose edilmez.

Internal Fraud Engine

- Tanım: FinWallet'ın kendi server-side verileriyle deterministic/rule-based risk kararı üreten bileşenidir.
- FinWallet'ta kullanım: Amount, velocity, new-device, beneficiary/merchant familiarity gibi sinyalleri değerlendirir.
- Neden / çözdüğü problem: External provider'a gitmeden açık riskleri yakalar ve domain-specific kontrol sağlar.
- Not / trade-off: Rule engine tek başına production ML fraud sistemi değildir; gerçek veriyle tuning gerekir.

Known Beneficiary

- Tanım: Transfer destination'ın müşteri tarafından daha önce kullanılan/tanınan beneficiary olup olmadığını belirten sinyaldir.
- FinWallet'ta kullanım: WalletTransfer risk context'inde kullanılabilir.
- Neden / çözdüğü problem: Yeni hedefe büyük transfer gibi daha riskli kombinasyonları ayırır.
- Not / trade-off: Beneficiary familiarity server-side history'den türetilmelidir; client boolean'ına güvenilmez.

Manual Fraud Review

- Tanım: Pending FraudEvent'in bir internal operasyon/reviewer tarafından approve veya deny edilmesidir.
- FinWallet'ta kullanım: /api/v1/internal/fraud-reviews Gateway InternalService policy arkasında çalışır.
- Neden / çözdüğü problem: Human-in-the-loop risk kararını müşteri admin user tipi yaratmadan güvenli internal boundary'de tutar.
- Not / trade-off: Reviewer identity ve reviewedAt audit kaydı gerekir; karar bir kez final olmalıdır.

Merchant Familiarity

- Tanım: Purchase merchant'ının müşteri geçmişinde tanidik olup olmadığını belirten risk sinyalidir.
- FinWallet'ta kullanım: Purchase fraud evaluation'da beneficiary yerine merchant context kullanılabilir.
- Neden / çözdüğü problem: Yeni merchant + yüksek amount gibi pattern'leri değerlendirmeyi sağlar.
- Not / trade-off: Tek başına fraud kanıtı değildir; diğer sinyallerle combine edilir.

New Device

- Tanım: Session/customer için daha önce görülmemiş device reference'ı olduğunu belirten risk sinyalidir.
- FinWallet'ta kullanım: Fraud context'te yeni cihaz işlemi daha riskli kabul edilebilir.
- Neden / çözdüğü problem: Credential ele geçirilmesi sonrası farklı cihazdan yapılan hareketi görünür kılar.
- Not / trade-off: Device fingerprint privacy ve spoofing riskleri nedeniyle tek başına deny sebebi olmamalıdır.

Reason Code

- Tanım: Fraud veya business kararının nedenini machine-readable kısa kodla açıklayan değerdir.
- FinWallet'ta kullanım: Internal/external fraud reason code'ları normalize edilip FraudEvent'e kaydedilir.
- Neden / çözdüğü problem: Operations, analytics ve testlerde karar nedenini text parse etmeden anlamayı sağlar.
- Not / trade-off: Client'a model bypass edecek kadar ayrıntılı risk rule bilgisi verilmemelidir.

Review

- Tanım: İşlemin otomatik allow/deny yerine manual review gerektirdiğini belirten fraud kararıdır.
- FinWallet'ta kullanım: FraudEvent durable Pending review state oluşturur ve public financial request 202 Accepted dönebilir.
- Neden / çözdüğü problem: Şüpheli ama kesin deny olmayan işlemleri para hareketi yapmadan bekletir.
- Not / trade-off: Review state durably saklanır; aynı idempotency request external fraud'u tekrar çağırmmalıdır.

Risk Signal

- Tanım: Fraud değerlendirmesinde kullanılan server-side davranış/bağlam verisidir.
- FinWallet'ta kullanım: Transaction count, 24h amount, country, device, known beneficiary ve merchant familiarity örnekleridir.
- Neden / çözdüğü problem: Client'ın 'ben güvenliyim' demesine güvenmek yerine risk context'i server'da üretir.

- Not / trade-off: Signal quality yanlışsa false-positive/false-negative artar.

Velocity

- Tanım: Kısa zaman aralığında yapılan işlem sayısı/tutar yoğunluğunu ölçen fraud sinyalıdır.
- FinWallet'ta kullanım: Son 5 dakikadaki transaction count ve son 24 saatteki amount gibi ölçüler kullanılır.
- Neden / çözdüğü problem: Account takeover veya automated abuse gibi hızlı davranışları yakalamaya yardımcı olur.
- Not / trade-off: Redis counter transient olabilir; final financial truth değildir.

8. Performans ve Gözlemlenebilirlik (13 terim)

Correlation / Traceability

- Tanım: Tek business flow'daki log, provider ve DB referanslarını birbirine bağlayabilme yeteneğidir.
- FinWallet'ta kullanım: CorrelationId, TransactionId, ExternalTransactionId, FraudReference ve Outbox/Inbox message ID birlikte kullanılır.
- Neden / çözdüğü problem: Distributed failure'ın hangi adımda oluştuğunu bulmayı kolaylaştırır.
- Not / trade-off: Tek bir universal ID her semantiği taşımaz; her reference'ın rolü ayrıdır.

Health Endpoint

- Tanım: Orchestrator/proxy/operator'ın servisin durumunu HTTP ile kontrol ettiği endpoint'tir.
- FinWallet'ta kullanım: Docker smoke test ve YARP health mekanizmaları /health/* endpoint'lerini kullanır.
- Neden / çözdüğü problem: Startup ve routing otomasyonunda service availability sinyali sağlar.
- Not / trade-off: Health endpoint hassas dependency/secret detail'i expose etmemelidir.

Latency

- Tanım: Bir request/operasyonun başlangıcından sonucuna kadar geçen süredir.
- FinWallet'ta kullanım: Gateway, provider HTTP, MSSQL ve Redis süreleri toplam API latency'yi oluşturur.
- Neden / çözdüğü problem: Kullanıcı deneyimi ve timeout tasarımı için temel metriktir.
- Not / trade-off: Sadece ortalama latency yeterli değildir; p95/p99 tail latency de izlenmelidir.

Liveness

- Tanım: Process'in çalışıyor ve deadlock/crash durumda olmadığını gösteren health kavramıdır.
- FinWallet'ta kullanım: Gateway/API live endpoint'i container'ın restart gerektirip gerektirmediğine sinyal verebilir.
- Neden / çözdüğü problem: Dependency geçici bozuldu diye process'i gereksiz restart etmekten kaçınmayı sağlar.
- Not / trade-off: Readiness ile ayrılması production orchestrator davranışını iyileştirir.

Log Rotation

- Tanım: Log dosyası/container logunun sınırsız büyümesini önlemek için boyut/sayı bazında döndürülmesidir.

- FinWallet'ta kullanım: Docker json-file logging için size/count sınırları compose'da verilir.
- Neden / çözdüğü problem: Disk dolması nedeniyle servis/host failure riskini azaltır.
- Not / trade-off: Central log retention'ın alternatifi değildir; local disk safety mekanizmasıdır.

Logical Reads

- Tanım: SQL query'nin buffer cache üzerinden kaç data page okuduğunu gösteren maliyet göstergesidir.
- FinWallet'ta kullanım: Index/query tuning kararlarında elapsed time yanında değerlendirilir.
- Neden / çözdüğü problem: Fast test ortamında bile gereksiz scan'i görünür kılar.
- Not / trade-off: Tek başına yeterli değildir; CPU, wait, row count ve plan ile birlikte okunmalıdır.

Observability

- Tanım: Sistemin iç durumunu log, metric, trace ve health sinyallerinden anlayabilme yeteneğidir.
- FinWallet'ta kullanım: FinWallet correlation IDs, structured logs, health endpoints, transaction/reconciliation records ile observability temeli kurar.
- Neden / çözdüğü problem: Incident ve performance analizinde 'neden oldu?' sorusunu cevaplamayı sağlar.
- Not / trade-off: Tam production observability için centralized log/metrics/tracing backend ayrıca gerekir.

p95 / p99

- Tanım: Request sürelerinin yüzde 95/99'unun altında kaldığı latency percentile ölçüleridir.
- FinWallet'ta kullanım: Performance review'unda hot endpoint/provider/SQL için tail latency değerlendirmesi önerilir.
- Neden / çözdüğü problem: Az sayıdaki çok yavaş request'in ortalamada gizlenmesini önler.
- Not / trade-off: Percentile anlamlı olmak için yeterli sample ve doğru time-window gerektirir.

Query Plan

- Tanım: SQL Server'ın bir query'yi hangi index/join/access yöntemleriyle çalıştıracağını gösteren execution plan'dır.
- FinWallet'ta kullanım: Performance tuning'de yeni index eklemekten önce plan ve logical reads incelenir.
- Neden / çözdüğü problem: Gerçek bottleneck'i tahmin yerine ölçmeye yardımcı olur.
- Not / trade-off: Plan environment/data distribution'a göre değişebilir; tek test database sonucu genellenmemelidir.

Query Store

- Tanım: SQL Server'ın query performance history, plans ve runtime statistics toplama özelliğidir.
- FinWallet'ta kullanım: Production-like tuning'de regressions ve expensive query'leri bulmak için önerilir.
- Neden / çözdüğü problem: Yeni index veya query değişikliğinin gerçek etkisini ölçmeyi kolaylaştırır.
- Not / trade-off: Storage/retention ayarları yönetilmelidir; uygulama logging'in yerine geçmez.

Readiness

- Tanım: Servisin yeni trafik almaya hazır olup olmadığını gösteren health kavramıdır.
- FinWallet'ta kullanım: MSSQL/Redis/schema init gibi dependency'ler hazır olmadan API'nin trafik almaması için kullanılabilir.
- Neden / çözdüğü problem: Startup race ve failed dependency durumunda request kaybını azaltır.
- Not / trade-off: Her dependency readiness'e dahil edilmemelidir; optional provider outage tüm servisi unready yapmak zorunda değildir.

Structured Logging

- Tanım: Log mesajını serbest text yerine isimli alanlarla üretme yaklaşımıdır.
- FinWallet'ta kullanım: CorrelationId, TransactionId, path, status ve safe error code gibi alanlar structured log olarak yazılabilir.
- Neden / çözdüğü problem: Search/aggregation ve incident investigation'ı kolaylaştırır.
- Not / trade-off: Password, OTP, token, connection string ve gereksiz PII structured field olarak loglanmaz.

Throughput

- Tanım: Bir sistemin belirli sürede işleyebildiği request/transaction sayısıdır.
- FinWallet'ta kullanım: Gateway, API, SQL pool ve provider capacity toplam throughput'u sınırlar.
- Neden / çözdüğü problem: Scale ve capacity planning için kullanılır.
- Not / trade-off: Yüksek throughput finansal correctness pahasına artırılmaz; lock/idempotency invariant'ları korunur.

9. Test ve Kalite (15 terim)

Chaos Test

- Tanım: Dependency delay/failure/restart gibi arızaları bilinçli üreterek sistem recovery davranışını test eder.
- FinWallet'ta kullanım: Fake provider fail/delay/timeout modları, container restart ve communication outage senaryoları buna temel oluşturur.
- Neden / çözdüğü problem: Fail-closed, outbox retry, blocked release ve reconciliation davranışını doğrular.
- Not / trade-off: Production chaos test kontrollü blast radius ve gözlemlenebilirlik gerektirir.

Concurrency Test

- Tanım: Aynı resource üzerinde eşzamanlı request'lerin invariant'ları bozmadığını doğrulayan testtir.
- FinWallet'ta kullanım: Örneğin 1000 balance için paralel iki 600 transferden yalnız birinin tamamlanması beklenir.
- Neden / çözdüğü problem: Overspend, idempotency race ve lock behavior'ını gerçek DB üzerinde kanıtlar.
- Not / trade-off: Thread-sleep tabanlı yapay test yerine gerçek paralel requests ve durable result doğrulaması gerekir.

Definition of Done

- Tanım: Bir feature'ın 'tamamlandı' sayılması için gereken kriterler bütünüdür.

- FinWallet'ta kullanım: Kod yanında API, DB, security, test, docs, config ve runtime doğrulaması güncellenmelidir.
- Neden / çözdüğü problem: Kod merge olup doküman/test geride kalması problemini azaltır.
- Not / trade-off: Checklist yaşayan bir standarttır; proje büyüdükçe yeni kalite kapıları eklenebilir.

End-to-End (E2E) Test

- Tanım: Client girişinden final durable sonucu kadar tüm kritik akışı gerçek component'lerle test eder.
- FinWallet'ta kullanım: Register -> login -> wallet -> bank movement -> transfer/purchase gibi happy/fail path'ler E2E olabilir.
- Neden / çözdüğü problem: Kullanıcı açısından sistemin gerçekten çalıştığını doğrular.
- Not / trade-off: Failure root cause'u unit test kadar hızlı lokalize etmez; daha az sayıda kritik senaryoya odaklanmalıdır.

Fixture

- Tanım: Testin ihtiyaç duyduğu önceden hazırlanmış veri veya environment state'idir.
- FinWallet'ta kullanım: Integration/E2E senaryolarında customer, wallet, bank account ve balance seed fixture olabilir.
- Neden / çözdüğü problem: Tekrarlanabilir senaryolar sağlar.
- Not / trade-off: Finansal fixture doğrudan balance update ile ledger'ı bozmak yerine dengeli seed mekanizması kullanılmalıdır.

Integration Test

- Tanım: Birden fazla gerçek component/dependency'nin birlikte çalışmasını test eder.
- FinWallet'ta kullanım: Docker stack üzerinde Gateway, FinWallet.Api, MSSQL, Redis ve fake provider zinciri integration test hedefidir.
- Neden / çözdüğü problem: DI, config, serialization, SQL schema ve network boundary hatalarını yakalar.
- Not / trade-off: Unit testten daha yavaş ve environment'a bağımlıdır.

Load Test

- Tanım: Sisteme kontrollü yüksek trafik vererek throughput, latency ve resource davranışını ölçen testtir.
- FinWallet'ta kullanım: Gateway/API/SQL/Redis pool ve rate-limit değerlerini production'a taşımadan önce doğrulamak için gerekir.
- Neden / çözdüğü problem: Config tuning'i tahmin yerine ölçüme bağlar.
- Not / trade-off: Financial test datası ve side effect'ler izole environment'ta tutulmalıdır.

Mock

- Tanım: Gerçek dependency yerine testin kontrol ettiği davranış sağlayan test double'dır.
- FinWallet'ta kullanım: Moq ile IBankProvider, store veya external dependency unit testlerde mocklanabilir.
- Neden / çözdüğü problem: Failure/edge-case davranışını hızlı ve deterministik üretir.
- Not / trade-off: Mock gerçek provider/SQL behavior'ını kanıtlamaz; fazla mock test'i implementation'a bağımlı hale getirebilir.

Moq

- Tanım: .NET unit testlerde interface/class mock üretmek için kullanılan mocking kütüphanesidir.
- FinWallet'ta kullanım: FinWallet test project'inde external/store dependency'leri izole etmek için kullanılır.
- Neden / çözdüğü problem: Setup/Verify ile expected interaction testini kolaylaştırır.
- Not / trade-off: Gerçek integration testin alternatifi değildir.

Smoke Test

- Tanım: Deploy/build sonrası sistemin temel olarak ayağa kalkıp kritik dependency'lere erişebildiğini hızlı kontrol eden testtir.
- FinWallet'ta kullanım: Docker CI tüm service'leri başlatıp Gateway health, MSSQL schema ve Redis connectivity doğrular.
- Neden / çözdüğü problem: YAML/build doğru olsa bile runtime DI/config/startup problemlerini erken yakalar.
- Not / trade-off: Derin business correctness testi değildir.

Strict Mock

- Tanım: Setup edilmeyen dependency çağrısı olduğunda testi fail eden mock davranışıdır.
- FinWallet'ta kullanım: Bazı handler testleri Moq Strict kullanarak beklenmeyen provider/store çağrısını yakalar.
- Neden / çözdüğü problem: Örneğin validation fail olduğunda bank provider'ın hiç çağrılmadığını kanıtlar.
- Not / trade-off: Çok sıkı testler refactor sırasında gereksiz kırılgan olabilir; business interaction için kullanılır.

Test Double

- Tanım: Gerçek dependency yerine testte kullanılan fake/mock/stub benzeri genel terimdir.
- FinWallet'ta kullanım: Unit testte provider/store davranışlarını kontrol etmek için kullanılır.
- Neden / çözdüğü problem: Testi hızlı ve deterministik yapar.
- Not / trade-off: Hangi double türünün kullanıldığı test niyetini açık tutmalıdır.

Unit Test

- Tanım: Tek sınıf veya küçük business behavior'ı dış dependency'lerden izole test eden test türüdür.
- FinWallet'ta kullanım: FinWallet.Application.Tests handler davranışlarını fake/mock dependency'lerle doğrular.
- Neden / çözdüğü problem: Hızlı feedback ve branch/error davranışını deterministik test etmeyi sağlar.
- Not / trade-off: Gerçek SQL locking, Redis atomicity veya Gateway routing'i kanıtlamaz.

Warnings as Errors

- Tanım: Compiler warning'lerini CI'da build failure olarak değerlendirme yaklaşımıdır.
- FinWallet'ta kullanım: Release build --warnaserror ile çalıştırılır.
- Neden / çözdüğü problem: Yeni warning'lerin sessizce birikmesini önler.
- Not / trade-off: Third-party/generated warning'ler gerektiğinde kontrollü suppression gerektirebilir.

xUnit

- Tanım: .NET için unit/integration test framework'üdür.
- FinWallet'ta kullanım: FinWallet.Application.Tests test runner/framework olarak xUnit v3 kullanır.
- Neden / çözdüğü problem: Fact/theory tabanlı test yazımı ve CI entegrasyonu sağlar.
- Not / trade-off: Test framework doğru test stratejisinin kendisi değildir; coverage ve scenario seçimi ayrıca tasarlanır.

10. Docker, CI/CD ve Konfigürasyon (25 terim)

.env

- Tanım: Docker Compose/local development için environment variable değerlerini tutan yerel dosyadır.
- FinWallet'ta kullanım: .env.example version-control'dadır, gerçek .env gitignore içindedir.
- Neden / çözdüğü problem: Local developer'ın gerekli config'i kolay hazırlamasını sağlar.
- Not / trade-off: Production secret-management yöntemi değildir.

appsettings.json

- Tanım: .NET configuration'ın temel JSON dosyasıdır.
- FinWallet'ta kullanım: Platform limitleri, provider URLs/timeouts, Redis/SQL tuning, Swagger ve Gateway config burada tanımlanır.
- Neden / çözdüğü problem: Operational değerleri koddan ayırır.
- Not / trade-off: Financial/cryptographic invariants config switch yapılmaz.

appsettings.Production.json

- Tanım: Production environment için base appsettings'i override eden standart .NET config dosyasıdır.
- FinWallet'ta kullanım: Swagger default-off, production URL/security ve boş secret placeholders gibi değerler taşır.
- Neden / çözdüğü problem: Development ile production davranışını source-code if'lerine boğmadan ayırır.
- Not / trade-off: Secret değerleri dosyaya commit edilmez; external injection beklenir.

BuildKit Cache

- Tanım: Docker build katman/restore çıktısını tekrar kullanarak sonraki image build'lerini hızlandıran cache mekanizmasıdır.
- FinWallet'ta kullanım: NuGet restore cache mount'ları .NET image build'lerinde kullanılır; parallel corruption riskine karşı locked sharing uygulanabilir.
- Neden / çözdüğü problem: CI build süresini azaltır.
- Not / trade-off: Cache correctness'ten önemli değildir; bozulursa clean build ile doğrulama yapılmalıdır.

CI (Continuous Integration)

- Tanım: Her değişikliği otomatik build/test/validation ile ana dala girmeden kontrol etme pratiğidir.
- FinWallet'ta kullanım: FinWallet CI Release build, warnings-as-errors ve tests çalıştırır; Docker workflow runtime doğrular.
- Neden / çözdüğü problem: Compile/runtime/schema regression'larını erken yakalar.

- Not / trade-off: CI yeşil olması production correctness'in tümünü garanti etmez; kritik E2E/security/load testleri de gerekir.

Compose Overlay

- Tanım: Ana compose config'e environment/debug-specific override ekleyen ek YAML dosyasıdır.
- FinWallet'ta kullanım: compose.debug.yml localhost debug portları açar; compose.production.yml production-like override sağlar.
- Neden / çözdüğü problem: Tek dosyada tüm environment conditional logic'i karıştırmadan config varyasyonu sağlar.
- Not / trade-off: Overlay sırası önemlidir; yanlış kombinasyon beklenmeyen port/setting açabilir.

Configuration Precedence

- Tanım: Aynı config key'in birden fazla kaynaktan geldiğinde hangisinin kazanacağını belirleyen sıradır.
- FinWallet'ta kullanım: Genel olarak appsettings -> environment-specific appsettings -> environment variables/secret injection üst üste uygulanır.
- Neden / çözdüğü problem: Aynı image'ı environment bazlı configure etmeyi sağlar.
- Not / trade-off: Resolved config çıktısı secret içerebileceği için loglanmamalıdır.

Container

- Tanım: Bir image'ın izole process, filesystem ve network namespace ile çalışan instance'ıdır.
- FinWallet'ta kullanım: Her FinWallet HTTP service ve MSSQL/Redis ayrı container olarak çalışır.
- Neden / çözdüğü problem: Dependency/version isolation ve hızlı recreate sağlar.
- Not / trade-off: Application container'ları stateless tasarlanır; durable data named volume/DB'dedir.

Docker

- Tanım: Uygulama ve dependency'leri izole container image/runtime olarak çalıştırma platformudur.
- FinWallet'ta kullanım: Gateway, FinWallet.Api, fake provider'lar, MSSQL ve Redis Docker Compose ile ayağa kaldırılabilir.
- Neden / çözdüğü problem: Local/integration environment'ın tekrar üretilebilir olmasını sağlar.
- Not / trade-off: Container kullanmak production orchestration/HA sorunlarını otomatik çözmez.

Docker Compose

- Tanım: Birden fazla container service, network ve volume'u tek YAML modelinde birlikte çalıştırma aracıdır.
- FinWallet'ta kullanım: compose.yml tüm FinWallet stack'ini tanımlar.
- Neden / çözdüğü problem: Local full-stack start/stop, dependency order ve config'i tek komutta yönetir.
- Not / trade-off: Production orchestrator'ın tam alternatifi olmak zorunda değildir.

Docker DNS / Service Discovery

- Tanım: Compose network içindeki service isimlerinin otomatik hostname olarak çözülmesidir.
- FinWallet'ta kullanım: Gateway destination'ları finwallet-api, fake-bank gibi service name'leri kullanabilir.
- Neden / çözdüğü problem: Hardcoded container IP ihtiyacını kaldırır.

- Not / trade-off: Service name environment dışında geçerli değildir; config ortam bazlı yönetilir.

Docker Network

- Tanım: Container'ların birbirini DNS/service name ile görebildiği izole sanal network'tür.
- FinWallet'ta kullanım: finwallet-backend HTTP servislerini; finwallet-data FinWallet.Api ile MSSQL/Redis'i bağlar.
- Neden / çözdüğü problem: DB/Redis'i public host network'e açmadan service connectivity sağlar.
- Not / trade-off: Network segmentation credential/auth yerine geçmez; service-key ve DB auth yine gerekir.

Dockerfile

- Tanım: Bir container image'ın nasıl build edileceğini tanımlayan declarative dosyadır.
- FinWallet'ta kullanım: docker/Dockerfile.webapi Gateway/API/fake servisler için ortak multi-stage build kullanır.
- Neden / çözdüğü problem: Build adımlarını version-control altında tekrarlanabilir hale getirir.
- Not / trade-off: Runtime image mümkün olduğunca küçük/non-root tutulmalıdır.

Environment Variable

- Tanım: Runtime config'i process environment üzerinden verme yöntemidir.
- FinWallet'ta kullanım: Connection string, password, JWT/service key ve tuning ayarları appsettings'i override edebilir.
- Neden / çözdüğü problem: Image/source değiştirmeden environment-specific config sağlar.
- Not / trade-off: Secret environment variable process inspection riskleri taşıyabilir; production secret store/orchestrator entegrasyonu tercih edilir.

GitHub Actions

- Tanım: GitHub repository event'lerinde otomatik CI/workflow çalıştıran platformdur.
- FinWallet'ta kullanım: Restore/build/test, Docker validation/smoke ve PDF generation workflow'ları kullanılır.
- Neden / çözdüğü problem: PR'a reproducible kalite kapısı ve artifact üretimi sağlar.
- Not / trade-off: Workflow permission ve secret kullanımında least privilege uygulanmalıdır.

Image

- Tanım: Container oluşturmak için kullanılan immutable filesystem/runtime paketidir.
- FinWallet'ta kullanım: .NET service image'ları multi-stage Dockerfile ile build edilir.
- Neden / çözdüğü problem: Aynı binary/runtime paketinin farklı environment'larda tutarlı çalışmasını sağlar.
- Not / trade-off: Secret veya mutable runtime data image içine bake edilmez.

Multi-Stage Build

- Tanım: Build araçları ile final runtime image'ını farklı Docker stage'lerinde ayıran tekniktir.
- FinWallet'ta kullanım: SDK stage restore/build/publish yapar; final ASP.NET runtime stage yalnız publish output taşır.
- Neden / çözdüğü problem: Final image boyutunu ve attack surface'i azaltır.

- Not / trade-off: Build cache ve project copy sırası iyi tasarlanmazsa CI yavaşlayabilir.

Named Volume

- Tanım: Docker'ın yaşam döngüsünü container'dan bağımsız yönettiği persistent storage alanıdır.
- FinWallet'ta kullanım: finwallet_mssql_data, finwallet_mssql_backup, finwallet_redis_data named volume'lardır.
- Neden / çözdüğü problem: Container recreate edildiğinde data'nın kalmasını sağlar.
- Not / trade-off: docker compose down -v volume'ları siler; destructive operasyon olarak dokümanite edilir.

Non-Root Container

- Tanım: Application process'inin container içinde root kullanıcısıyla çalışmamasıdır.
- FinWallet'ta kullanım: .NET runtime image USER app yaklaşımıyla çalıştırılır.
- Neden / çözdüğü problem: Container escape/compromise durumunda privilege impact'i azaltır.
- Not / trade-off: Bazı filesystem/port işlemleri için permission planı gerekir.

Read-Only Filesystem

- Tanım: Application container filesystem'inin runtime'da yazılamaz hale getirilmesi hardening yaklaşımıdır.
- FinWallet'ta kullanım: Stateless API container'larında mümkün olduğunca read-only root filesystem kullanılabilir.
- Neden / çözdüğü problem: Malware/config tampering ve accidental local state riskini azaltır.
- Not / trade-off: Temp/runtime write ihtiyacı olan path'ler ayrı tmpfs/volume gerektirebilir.

Resource Limit

- Tanım: Container'ın CPU, memory ve PID kullanımına üst sınır koymaktır.
- FinWallet'ta kullanım: Compose application container'larında local safety baseline limitleri tanımlanabilir.
- Neden / çözdüğü problem: Tek bozuk servisin host resource'larını tüketmesini sınırlar.
- Not / trade-off: Production limitleri load test ve capacity ölçümüyle belirlenmelidir.

SBOM (Software Bill of Materials)

- Tanım: Bir build/artifact içinde hangi software dependency ve versionların bulunduğunu listeleyen envanterdir.
- FinWallet'ta kullanım: Security dokümanında supply-chain improvement olarak değerlendirilir; v1'in temel runtime component'i değildir.
- Neden / çözdüğü problem: Vulnerability/incident sırasında hangi artifact'ın etkilendiğini hızlı bulmayı sağlar.
- Not / trade-off: SBOM üretmek vulnerability'yi otomatik çözmez; scan/remediation workflow'u gerekir.

Supply Chain Security

- Tanım: Build/package/dependency zincirinin kötü niyetli veya vulnerable bileşenlerden korunmasıdır.
- FinWallet'ta kullanım: Central NuGet versions, explicit dependencies, CI restore/build/test ve planlanan vulnerability/SBOM kontrolleri bu alana girer.

- Neden / çözdüğü problem: Kod doğru olsa bile third-party dependency kaynaklı riskleri azaltır.
- Not / trade-off: Sadece package pinlemek yeterli değildir; provenance/vulnerability/update süreci gerekir.

Vulnerability Scan

- Tanım: Dependency, container image veya code üzerinde bilinen güvenlik açıklarını otomatik kontrol etme işlemidir.
- FinWallet'ta kullanım: FinWallet CI hardening roadmap'ında supply-chain kalite kapısı olarak yer alabilir.
- Neden / çözdüğü problem: Known-CVE risklerini merge/deploy öncesi görünür kılar.
- Not / trade-off: False positive ve exploitability context'i için human review gerekebilir.

Workflow Artifact

- Tanım: CI run sırasında üretilen ve indirilebilir saklanan dosya paketidir.
- FinWallet'ta kullanım: Generated TR/EN PDF setleri GitHub Actions artifact olarak da upload edilir.
- Neden / çözdüğü problem: Binary çıktıyı merge öncesi inspect/render etmeyi kolaylaştırır.
- Not / trade-off: Artifact retention geçicidir; final versioned PDF'ler repo içinde ayrıca commit edilir.

11. .NET ve Uygulama Kodlama (15 terim)

.NET 8

- Tanım: FinWallet'ın hedef runtime/framework sürümüdür.
- FinWallet'ta kullanım: Tüm ana API ve fake provider projeleri .NET 8 ile build edilir.
- Neden / çözdüğü problem: Modern ASP.NET Core, built-in DI, rate limiting, HttpClientFactory ve container desteği sağlar.
- Not / trade-off: Framework upgrade'leri ayrı compatibility/test çalışması gerektirir.

ASP.NET Core

- Tanım: .NET üzerinde HTTP API ve web uygulaması geliştirme framework'üdür.
- FinWallet'ta kullanım: FinWallet.Gateway, FinWallet.Api ve simulator API'leri ASP.NET Core kullanır.
- Neden / çözdüğü problem: Kestrel, middleware, authentication, controllers, DI ve config altyapısını sağlar.
- Not / trade-off: Business domain'i framework API'lerine bağımlı tutulmaz.

async/await

- Tanım: .NET'te I/O beklerken thread'i bloklamadan asynchronous programlama yapma modelidir.
- FinWallet'ta kullanım: HTTP, SQL, Redis ve background worker I/O akışları async çalışır.
- Neden / çözdüğü problem: Yük altında thread-pool starvation riskini azaltır.
- Not / trade-off: Async, işlemi otomatik paralel veya transactional yapmaz.

Built-in DI Container

- Tanım: ASP.NET Core'un kendi dependency injection container'ıdır.
- FinWallet'ta kullanım: FinWallet üçüncü parti DI container yerine IServiceCollection registration kullanır.

- Neden / çözdüğü problem: Dependency graph'i standart framework mekanizmasıyla yönetir.
- Not / trade-off: Çok ileri interception/child-container ihtiyacı olmadıkça ekstra container eklenmez.

CancellationToken

- Tanım: .NET async operasyonlarında caller request iptalini alt dependency'lere ileten cancellation sinyalıdır.
- FinWallet'ta kullanım: Controller -> handler -> SQL/Redis/HttpClient çağrılarına propagate edilir.
- Neden / çözdüğü problem: Client bağlantısı koptuğunda gereksiz I/O'nun devam etmesini azaltır.
- Not / trade-off: Committed financial transaction cancellation ile 'undo' edilmez; cancellation yalnız commit öncesi/ongoing work'u durdurur.

Central Package Management

- Tanım: NuGet package versionlarının her csproj yerine merkezi dosyadan yönetilmesidir.
- FinWallet'ta kullanım: Directory.Packages.props gibi yapı ile xUnit, Moq, YARP, Swashbuckle vb. versiyonlar merkezi pinlenir.
- Neden / çözdüğü problem: Solution içinde version drift ve duplicate version tanımlarını azaltır.
- Not / trade-off: Major upgrade yine compatibility review gerektirir; merkezi pin otomatik güvenlik sağlamaz.

Controller-Based API

- Tanım: ASP.NET Core endpoint'lerini attribute/routing kullanan Controller sınıflarıyla tanımlama yaklaşımıdır.
- FinWallet'ta kullanım: FinWallet Minimal API yerine controllers kullanır.
- Neden / çözdüğü problem: Büyük API surface'te response metadata, authorization ve organization'ı daha görünür tutar.
- Not / trade-off: Küçük endpointlerde daha fazla boilerplate olabilir.

Exception Mapping

- Tanım: Application/domain exception'larını kontrollü HTTP status + machine code response'a dönüştürme katmanıdır.
- FinWallet'ta kullanım: Insufficient balance, fraud deny/review, idempotency conflict, provider unavailable gibi expected exception'lar merkezi API mapping ile ServiceResult'a çevrilir.
- Neden / çözdüğü problem: Business error'ın 500 internal error olarak sızmasını önler.
- Not / trade-off: Unexpected exception delay/stack trace client'a verilmez; loglanır ve generic 500 döner.

Immutable Model

- Tanım: Oluşturulduktan sonra key financial history alanlarının değiştirilmemesi yaklaşımıdır.
- FinWallet'ta kullanım: Ledger entries/journals ve completed transaction history correction ile overwrite edilmez.
- Neden / çözdüğü problem: Audit trail ve reasoning'i korur.
- Not / trade-off: Current-state projection'lar (Wallet balance) mutable olabilir; history ile projection ayrımı önemlidir.

Machine-Readable Error Code

- Tanım: Client/operation logic'in free-text message yerine stabil kodla hata türünü ayırt etmesini sağlayan string'dir.
- FinWallet'ta kullanım: IDEMPOTENCY_CONFLICT, FRAUD_REVIEW_REQUIRED, RATE_LIMITED gibi kodlar kullanılır.
- Neden / çözdüğü problem: Localization/message değişse bile client branching'i stabil tutar.
- Not / trade-off: Aynı code semantiği farklı endpointlerde çelişkili kullanılmamalıdır; error-code kataloğu bu nedenle tutulur.

Middleware

- Tanım: HTTP request/response pipeline'ında controller'dan önce/sonra çalışan cross-cutting bileşendir.
- FinWallet'ta kullanım: Shared.Web security headers, method/content checks, correlation ve rate-limit pipeline davranışlarını uygular.
- Neden / çözdüğü problem: Her controller'da aynı platform kontrolünü tekrar yazmayı önler.
- Not / trade-off: Business use-case logic middleware'e taşınmaz.

NuGet

- Tanım: .NET package/dependency dağıtım ekosistemidir.
- FinWallet'ta kullanım: YARP, Swashbuckle, Moq, xUnit ve SQL/Redis client gibi external packages NuGet üzerinden alınır.
- Neden / çözdüğü problem: Tekrar kullanılabilir library/tooling entegrasyonunu kolaylaştırır.
- Not / trade-off: Supply-chain riski nedeniyle package sayısı, lisans ve vulnerability durumu kontrol edilmelidir.

TimeProvider

- Tanım: .NET'te zamanı doğrudan DateTime.UtcNow çağırmak yerine inject edilebilir abstraction üzerinden alma mekanizmasıdır.
- FinWallet'ta kullanım: Fraud evaluation/review timestamp gibi Application logic'inde kullanılır.
- Neden / çözdüğü problem: Time-dependent testleri deterministik hale getirir.
- Not / trade-off: DB/provider authoritative time ile application time farkları ayrıca düşünülmelidir.

Typed HttpClient

- Tanım: Belirli provider için HttpClient configuration ve adapter logic'ini strongly-typed class ile birleştiren .NET pattern'idir.
- FinWallet'ta kullanım: Bank/Fraud/Cutoff/Campaign/Communication provider client'ları typed HttpClient olarak DI'a bağlanır.
- Neden / çözdüğü problem: Base URL, timeout, handler ve provider API çağrılarını tek boundary'de tutar.
- Not / trade-off: Client sınıfı domain business orchestration yapmamalıdır.

XML Documentation Comments

- Tanım: C# public type/method/property'lerde IDE/Swagger/doküman için kullanılan /// <summary> yorum formatıdır.
- FinWallet'ta kullanım: FinWallet public class/interface/metodlarda TR ve EN açıklama standardı uygular.
- Neden / çözdüğü problem: Yeni geliştiricinin API/code intent'ini hızlı anlamasını sağlar.

- Not / trade-off: Yorum kodla güncel tutulmazsa yanlış bilgi üretir; Definition of Done'a dahildir.

12. Bilinçli Olarak Kullanılmayan / Yerine Başka Yaklaşım Seçilen Terimler (11 terim)

ASP.NET Core Identity

- Tanım: Microsoft'un hazır user, password, token, role ve auth persistence framework'üdür.
- FinWallet'ta kullanım: FinWallet custom auth/session/refresh tasarımı nedeniyle kullanılmaz.
- Neden / çözdüğü problem: Proje bankacılık benzeri custom session, refresh rotation, OTP ve explicit schema davranışını öğrenme/kontrol amacıyla kendisi uygular.
- Not / trade-off: Gerçek production üründe hazır ve battle-tested identity solution kullanmak bazı riskleri azaltabilir; custom auth daha fazla security sorumluluğu getirir.

Blind Retry

- Tanım: Operasyonun side effect/idempotency semantiğini bilmeden otomatik tekrar denemektir.
- FinWallet'ta kullanım: Financial provider POST'larında kullanılmaz.
- Neden / çözdüğü problem: İlk request provider'da başarılı ama response kaybolmuşsa ikinci retry duplicate para hareketi yaratabilir.
- Not / trade-off: Retry ancak stable provider request key/idempotency ve operation status semantics ile güvenli hale getirilir.

CQRS

- Tanım: Command/write modeli ile query/read modelini ayrı model veya pipeline'lara bölme yaklaşımıdır.
- FinWallet'ta kullanım: FinWallet tam CQRS framework uygulamaz; bazı write handler ve read history handler ayrımları doğal olarak vardır.
- Neden / çözdüğü problem: V1'de ekstra mediator/read-store complexity yerine sade Application handlers tercih edilir.
- Not / trade-off: İleride read scale veya model divergence artarsa CQRS daha anlamlı olabilir.

Direct Balance UPDATE

- Tanım: Test veya operasyon kolaylığı için wallet balance kolonunu ledger/transaction olmadan elle değiştirmektir.
- FinWallet'ta kullanım: FinWallet happy-path dokümanlarında özellikle yapılmaması belirtilir.
- Neden / çözdüğü problem: Wallet projection ile Ledger'ı ayırarak reconciliation mismatch ve açıklanamayan para yaratır.
- Not / trade-off: Test funding bile dengeli provider/ledger fixture veya resmi financial posting akışıyla yapılmalıdır.

Distributed Transaction

- Tanım: Birden fazla database/service'in tek atomik commit gibi koordine edilmesi problemdir.
- FinWallet'ta kullanım: FinWallet external HTTP çağrısını MSSQL transaction içine almaz ve 2PC benzeri distributed transaction kurmaz.
- Neden / çözdüğü problem: Lock/availability/operational complexity'i azaltmak için local ACID + idempotency + compensation + reconciliation kullanılır.

- Not / trade-off: Dış sistemle tam eşzamanlı atomicity yerine explicit eventual consistency kabul edilir.

Event Sourcing

- Tanım: Current state'i doğrudan saklamak yerine tüm domain event'lerini source of truth olarak saklayıp state'i event'lerden türetme yaklaşımıdır.
- FinWallet'ta kullanım: FinWallet'ta kullanılmaz; immutable double-entry Ledger history vardır ama domain'in tamamı Event Sourcing değildir.
- Neden / çözdüğü problem: Ledger auditability sağlarken Wallet current projection ve normal relational tables pratik sorgu/işletim kolaylığı sağlar.
- Not / trade-off: Ledger ile Event Sourcing aynı şey değildir.

Generic Repository

- Tanım: Her entity için aynı CRUD interface'ini sunan genel repository abstraction'ıdır.
- FinWallet'ta kullanım: FinWallet finansal store'larda kullanılmaz; use-case'e özel IWalletTransferPostingStore, IFraudEventStore gibi boundary'ler tercih edilir.
- Neden / çözdüğü problem: Transaction/locking/idempotency gibi financial semantics'i generic CRUD arkasında gizlememeyi sağlar.
- Not / trade-off: Basit CRUD alanlarında generic abstraction mümkün olsa da financial hot-path için uygun görülmemiştir.

MediatR

- Tanım: .NET'te request/handler dispatch için yaygın mediator kütüphanesidir.
- FinWallet'ta kullanım: FinWallet bilinçli olarak kullanmaz; Controller handler'ı DI üzerinden doğrudan çağırır.
- Neden / çözdüğü problem: Call path'i görünür tutar ve gereksiz abstraction/package katmanını azaltır.
- Not / trade-off: Cross-cutting pipeline ihtiyacı büyürse mediator tekrar değerlendirilebilir.

Microservices

- Tanım: Her domain/servisin bağımsız deploy/scale edilebildiği dağıtık mimari yaklaşımıdır.
- FinWallet'ta kullanım: FinWallet finansal çekirdekte v1 için bilinçli olarak kullanılmaz; fake external providers ayrı HTTP servisleri olarak tutulur.
- Neden / çözdüğü problem: Wallet/Ledger/Transaction gibi aynı atomik commit gerektiren alanları erken bölmek distributed consistency maliyetini azaltır.
- Not / trade-off: İleride bağımsız scale/team ownership ihtiyacı gerçek olduğunda sınırlar yeniden değerlendirilebilir.

Redis as Financial Source of Truth

- Tanım: Wallet/ledger gibi kalıcı finansal state'i sadece Redis'te tutma yaklaşımıdır.
- FinWallet'ta kullanım: FinWallet bilinçli olarak kullanmaz.
- Neden / çözdüğü problem: Redis restart/eviction/operational behavior'ının para doğruluğunu belirlemesini engellemek için MSSQL authoritative tutulur.
- Not / trade-off: Redis performans destek katmanıdır; financial truth ve audit history değildir.

Two-Phase Commit (2PC)

- Tanım: Birden fazla resource manager'ın prepare/commit aşamalarıyla distributed transaction koordine ettiği protokoldür.
- FinWallet'ta kullanım: FinWallet v1 bank/provider entegrasyonunda kullanılmaz.
- Neden / çözdüğü problem: External provider'ların 2PC desteklememesi ve availability/complexity maliyeti nedeniyle local transaction pattern'leri tercih edilir.
- Not / trade-off: Banking entegrasyonlarında çoğu gerçek HTTP provider zaten distributed XA participant değildir.